

Performance–Security Evaluation of Layer-1 Consensus Algorithms under Network Delay and Adversarial Conditions: A Simulation-Based Comparative Study

by

Le Ngoc Phan Anh

Submitted to the Department of Information and Communication Technology
in partial fulfillment of the requirements for the degree of

BACHELOR OF INFORMATION AND COMMUNICATION TECHNOLOGY

at the

UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

July 2026

© 2026 Le Ngoc Phan Anh. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by: Le Ngoc Phan Anh
Department of Information and Communication Technology
July 2026

Certified by: Dr. Giang Anh Tuan
Thesis Supervisor, Thesis Supervisor

Accepted by: [Acceptor Name TBD]
[Title TBD]
Department of Information and Communication Technology

THESIS COMMITTEE

THESIS SUPERVISOR

Dr. Giang Anh Tuan

Thesis Supervisor

Department of Information and Communication Technology

THESIS READERS

[Reader 1 Name TBD]

[Title TBD]

Department of Information and Communication Technology

[Reader 2 Name TBD]

[Title TBD]

Department of Information and Communication Technology

Performance–Security Evaluation of Layer-1 Consensus Algorithms under Network Delay and Adversarial Conditions: A Simulation-Based Comparative Study

by

Le Ngoc Phan Anh

Submitted to the Department of Information and Communication Technology
on July 2026 in partial fulfillment of the requirements for the degree of

BACHELOR OF INFORMATION AND COMMUNICATION TECHNOLOGY

ABSTRACT

The developments of the “kinetic theory” of gases made within the last ten years have enabled it to account satisfactorily for many of the laws of gases. The mathematical deductions of Clausius, Maxwell and others, based upon the hypothesis of a gas composed of molecules acting upon each other at impact like perfectly elastic spheres, have furnished expressions for the laws of its elasticity, viscosity, conductivity for heat, diffusive power and other properties. For some of these laws we have experimental data of value in testing the validity of these deductions and assumptions. Next to the elasticity, perhaps the phenomena of the viscosity of gases are best adapted to investigation.¹

Thesis supervisor: Dr. Giang Anh Tuan

Title: Thesis Supervisor

¹Text from Holman (1876): doi:[10.2307/25138434](https://doi.org/10.2307/25138434).

Acknowledgments

I am grateful to my thesis advisor, Dr. Giang Anh Tuan, for guidance and supervision throughout this work, and for the feedback that shaped its direction and methodology. I also thank TODO(human: committee members or thesis readers, with titles) for their time and comments on the work, and the faculty and staff of the Department of Information and Communication Technology at the University of Science and Technology of Hanoi for the instruction and support that made it possible.

I thank TODO(human: fellow students, collaborators, or colleagues) for the discussions and feedback that improved this thesis. This work was supported by TODO(human: funding source, fellowship, or grant number — delete this sentence if no such support applies), and I am grateful for the resources and compute that support provided.

Finally, I thank TODO(human: family members, partner, and friends) for their patience, encouragement, and support throughout my studies.

Contents

<i>List of Figures</i>	8
<i>List of Tables</i>	11
1 Introduction	12
1.1 Background	12
1.2 Motivation	12
1.3 Problem statement	13
1.4 Scope and assumptions	13
1.5 Research questions	14
1.6 Contributions and thesis roadmap	15
2 Literature Review	16
2.1 Blockchains and the consensus problem	16
2.2 The fork in the road	17
2.3 The three families	17
2.3.1 PBFT-style	19
2.3.2 PoS-finality	19
2.3.3 Avalanche-style	19
2.4 Why the published numbers do not compare	19
2.4.1 Each family reports in its own vocabulary	19
2.4.2 The surveys measure nothing, and the one precedent targets PoW	20
2.5 The gap	20
3 Methodology	22
3.1 Overview	22
3.2 System model	22
3.3 Algorithms	26
3.3.1 PBFT	27
3.3.2 Casper FFG	29
3.3.3 Snowman	30
3.4 Simulation setup	31
3.4.1 Reproducibility and the run lifecycle	31
3.4.2 The experiment matrix	32
3.4.3 Regime-coherence constraints	33
3.4.4 One run, end to end	34

3.5	Metric schema	37
3.6	Summary and threats to validity	40
4	Results	42
4.1	Chapter roadmap	42
4.2	Baseline: scaling with validator-set size	42
4.2.1	Statistical reliability	43
4.2.2	Latency	43
4.2.3	Throughput and goodput	44
4.2.4	Communication overhead	46
4.2.5	Reliability	48
4.2.6	A note on the latency measurement point	49
4.2.7	Baseline summary	50
4.3	Network-delay sweep	50
4.3.1	Delay and time-to-finality	50
4.3.2	Packet loss and the resilience ranking	52
4.3.3	Mechanisms of degradation	53
4.3.4	The latency–liveness tradeoff	55
4.4	Adversarial sweep	56
4.4.1	Delayed voting	57
4.4.2	Silent non-participation	58
4.4.3	Equivocation	59
4.4.4	The performance–security tradeoff	62
5	Synthesis	66
5.1	Chapter roadmap	66
5.2	The comparison and its axes	66
5.3	Where each family sits on the frontier	67
5.3.1	PBFT: fast and live, but the fork is unaccountable	67
5.3.2	Casper FFG: never fastest, but the only accountable failure	68
5.3.3	Snowman: safest under equivocation, costliest under delay	68
5.4	The frontier and the no-dominance verdict	69
5.5	Implications and hand-off	72
6	Conclusions	73
6.1	Summary of findings	73
6.2	Limitations	74
6.3	Directions for further work	75
6.3.1	Production-optimized protocol variants	75
6.3.2	Further directions	76
6.4	Concluding remarks	76
A	Code listing	78

B One-term coefficients for heat conduction	80
B.1 A multipage table of numbers	80
<i>References</i>	82

List of Figures

2.1	From the Byzantine Generals Problem to the three families.	17
3.1	Structural view: a single fixed harness in which only the protocol-logic slot is swapped, turning one experiment-matrix cell and seed into one comparable per-trial row.	23
3.2	One turn of the scheduler’s event loop — pop the next event, advance the virtual clock to it, let a node react, enqueue what it produces — with the three termination paths.	25
3.3	PBFT three-phase commit for one (view , seq) instance with the view-change branch.	28
3.4	Casper FFG justify→finalize for one epoch with the slashing branch.	29
3.5	Snowman subsampled K -peer poll loop for one block, accepting at counter $\geq \beta$	30
3.6	One seeded run as a temporal sequence of the six phases — init, workload, run loop, stop, flush, output — producing one results.csv row, with the run-loop branch showing where the delay (Family B) and adversarial (Family C) sweeps diverge from the honest baseline (Family A). The adversary is drawn inside the Validator lane, not as a separate component: it is the per-node interceptor of §3.2 that alters a bound validator’s outgoing messages.	35
4.1	Goodput versus validator-set size with 95% confidence intervals — the one baseline metric carrying a non-degenerate interval.	43
4.2	Commit latency versus validator-set size.	44
4.3	Decision rate (tps) versus validator-set size.	45
4.4	Goodput versus validator-set size, flat in n	46
4.5	Message overhead per committed unit versus validator-set size, logarithmic axis.	47
4.6	Measured message overhead against predicted asymptotic cost. Markers are the simulator’s measured total_msgs_per_acu ; dashed lines are the per-protocol predictions — PBFT $2n$, Casper FFG $1.2n$, and Snowman $2 \cdot K \cdot \beta$ with $K = \min(20, n-1)$. Vertical axis logarithmic. The overlay is the visual form of the theory-match claim made in this section; the residual gaps at $n = 4$ are the finite- n corrections.	48
4.7	Reliability: success rate and fork rate versus validator-set size.	49
4.8	Commit latency under moderate delay. Median per-validator commit_latency_ms under the two equal-mean moderate timelines (delay-uniform and delay-exponential), grouped by protocol and faceted by validator count; logarithmic vertical axis.	51

4.9	Finalization rate under packet loss. Finalization rate against per-message drop probability, one curve per protocol, faceted by validator count, with 95% confidence intervals.	53
4.10	Loss-resilience ranking. Area under the finalization-rate curve (AURC) with 95% confidence intervals, annotated with each cell's survival depth p^* , faceted by validator count; protocols whose intervals overlap share a rank.	53
4.11	Commit-latency growth under loss. <code>commit_latency_ms</code> against drop probability on a logarithmic axis; a solid line marks the levels at which a protocol still finalizes and a cross marks the level at which liveness is lost.	54
4.12	Communication cost of survival. Messages per committed unit against drop probability on a logarithmic axis, with PBFT's view-change counts annotated.	55
4.13	The operator tradeoff. Finalization rate retained against the added-latency ratio (loss latency divided by control latency, logarithmic), faceted by validator count; cells that no longer finalize are pinned on a "no finality" band, and the operator-best region is the upper left.	56
4.14	Liveness and time-to-finality under delayed voting. Top row: finalization success rate against the injected adversarial fraction φ , faceted by validator count, on which PBFT and Snowman coincide at 1.0 while Casper FFG dips. Bottom row, on a logarithmic scale: the time-to-finality ratio against the honest control, which separates the two protocols the top row leaves indistinguishable — PBFT and Casper FFG hold at $1.0\times$ while Snowman pays a blow-up of roughly $\times 62$ at $n = 10$ and $\times 49$ at $n = 25$	58
4.15	Liveness under silent non-participation. Finalization success rate against the injected silent fraction φ , drawn as step cliffs, one curve per protocol and faceted by validator count, with each protocol's survival depth φ^* — the deepest fraction at which it still finalizes — boxed. The cliffs place PBFT and Casper FFG ahead of Snowman; the Snowman cell that remains alive at $n = 25$, $\varphi = 0.20$ is labelled to mark that it finalizes only at a collapsed fraction of its control throughput.	59
4.16	Liveness under equivocation. Finalization success rate against the equivocator fraction φ , one curve per protocol, faceted by validator count; PBFT's curve is non-monotone, its apparent recovery above $\varphi = 0.33$ being the safety failure of Figure 4.18, not restored liveness.	61
4.17	PBFT view-change count under equivocation. Number of view-changes against the equivocator fraction φ , on a count axis shared across the two committee sizes so the size effect is visible; the count rises to its plateau — ten at $n = 10$ and twenty-five at $n = 25$ — at the last safe fraction and collapses to zero at $\varphi = 0.40$, where the deterministic fork replaces leader rotation.	61
4.18	Cross-protocol safety-violation rate under equivocation. Safety-violation rate against the equivocator fraction φ , drawn as steps, one curve per protocol and faceted by validator count; only PBFT departs from zero, stepping to a deterministic fork at $\varphi = 0.40$, where the magnitude of the fork — 229 conflicting (<code>view</code> , <code>seq</code>) instances at both committee sizes — is annotated. Casper FFG and Snowman stay at zero; their failure modes appear on the stake axis of Figure 4.19 and through ε respectively.	62

4.19	Casper FFG slashable stake under equivocation. Maximum slashable stake fraction against the equivocator fraction φ , faceted by validator count, with the one-third accountability line marked.	62
4.20	Throughput degradation versus adversarial fraction (silent non-participation). Committed-unit throughput against the injected silent fraction φ for each protocol, one panel per committee size ($n = 10$, left; $n = 25$, right); the three RQ2 degradation modes read off the curves directly — PBFT undegraded to its quorum cliff, Casper FFG decaying in proportion to the participating stake ($\approx 1 - \varphi$), and Snowman starving earliest.	64
4.21	Adversarial outcomes by protocol and strategy. The nine protocol–strategy cells of Table 4.2 rendered as an outcome map: the cell color encodes the kind of outcome — robust with liveness held, survival at a latency cost, liveness loss, accountable safety failure, or unaccountable safety break — and each cell label carries the governing magnitude. No protocol occupies a single color across its row: PBFT runs from robust under the two liveness adversaries to an unaccountable break under equivocation, Snowman from costly survival under delay through liveness loss under silence to robust under equivocation, and Casper FFG is never first yet never catastrophic. The image carries the no-dominance verdict and the structural inversions that produce it.	65
5.1	Cross-family performance–security frontier. The three families evaluated, scored on the eight cross-regime axes of Table 5.1 and normalized by ordinal rank per axis: the outer ring marks the strict best on an axis and the center the worst, with ties shared. The polygons overlap and none encloses another. Each family reaches the outer ring on at least one axis no other matches — PBFT on the delay, loss, and liveness axes, Casper FFG on communication overhead and accountable safety, Snowman on equivocation safety — so each is non-dominated and no family dominates, the verdict of Table 5.1 read directly off one image. The rank scale sets aside the magnitudes that drive it, which are the Chapter 4 headlines: Snowman’s time-to-finality grows roughly sixty-twofold under delayed voting and its polling overhead reaches about fourteen times PBFT’s, PBFT forks into 229 conflicting committed instances past its threshold, and Snowman’s analytical safety bound is near 5×10^{-15} while Casper FFG exposes at least one-third of stake as slashable. Two axes are not symmetric contests, as in Table 5.1: equivocation safety ranks Snowman first on its reported analytical bound rather than an empirical witness, and accountable safety names a capability only a slashing-based protocol offers. .	71

List of Tables

2.1	How one block becomes irreversible	18
2.2	Reported metric vocabulary across families	20
3.1	Validator capabilities and their owners	24
3.2	Family-to-protocol mapping	26
3.3	The implemented protocols in the simulator	27
3.4	Shape of one comparative-CSV row	36
3.5	Latency and throughput per protocol	38
3.6	Overhead and reliability per protocol	39
4.1	Baseline means at $n = 25$	50
4.2	Adversarial outcomes by protocol and strategy	63
5.1	Cross-regime comparison of the three families	70
6.1	The five research questions and their answers	73
B.1	One-term coefficients for one-dimensional heat conduction with a convective boundary condition. Data follow H. D. Baehr and K. Stephan [baehr1998].	80

Chapter 1

Introduction

1.1 Background

A Layer-1 (L1) blockchain maintains a replicated, append-only log of transactions across a set of mutually distrusting validators. At each block height the validator set must agree on a single block; the protocol that produces that agreement is the consensus protocol.

Three foundational results bound any such protocol. Deterministic Byzantine agreement requires $n \geq 3f + 1$ replicas [1]. Under pure asynchrony no deterministic protocol guarantees both safety and liveness against even one crash fault, by the FLP impossibility [2]. Under partial synchrony consensus becomes solvable for $f < n/3$ [3]. Every deployed L1 protocol sits at some point in the design space these results define.

Three families occupy distinct points in that space, each trading a different cost for its guarantees:

- **PBFT-style** — leader-driven multi-phase voting; deterministic finality at commit; cost: $O(n^2)$ messages, which bounds the validator-set size.
- **PoS-finality** — a stake-weighted finality gadget over epoch checkpoints; deterministic finality; cost: finality latency on the order of minutes.
- **Avalanche-style** — repeated random subsampling; probabilistic finality with tunable confidence; cost: no deterministic safety.

How these mechanisms differ in operation is developed in Chapters 2 and 3; quantifying the cost differences is the task of this thesis.

1.2 Motivation

In September 2021 the Solana mainnet stopped producing blocks for seventeen hours after a flood of bot transactions overwhelmed the network [4]. The halt was not an isolated event. Solana stalled again in April 2022 from validator memory exhaustion compounded by insufficient finalization votes, in February 2023 from block-propagation saturation, and in February 2024 from a five-hour finalization stall [4]. Ethereum, secured by a PoS-finality

protocol [5], experienced a seven-block reorganization in May 2022 and a multi-epoch finality stall in May 2023 driven by attestation-processing pressure [6,7]. The Cosmos Hub, secured by a PBFT-style protocol [8], halted after its v17.1 upgrade in June 2024 [9]. Sui suffered a crash-loop network halt in November 2024 [10]. The Avalanche mainnet runs the protocol from which its family takes its name [11].

These incidents do not invalidate the protocols’ theoretical guarantees. They demonstrate that the conditions under which those guarantees hold — bounded delay, a sufficiently honest validator set — are routinely exited in deployment. Isolating which condition triggers which failure requires controlled measurement that live networks do not afford. The primary literature does not supply it: each study fixes its own network model, fault assumptions, and workload, and varies one disturbance at a time [12,13]. Live operation combines those disturbances. Validators are geographically distributed; messages are delayed, reordered, and dropped; and a fraction of the active set is, at any moment, slow, offline, or adversarial.

Under those combined conditions, performance and security cease to be independent. A protocol that only *slows* under load may also miss finality, fork, or admit conflicting commits across honest validators. This coupling is invisible to the benign-condition benchmarks each family publishes. This thesis is motivated by that coupling and by the absence of a unified harness in which it can be measured across the three families on matched assumptions.

1.3 Problem statement

This thesis investigates the comparative performance–security behavior of the three L1 consensus families — PBFT-style, PoS-finality, and Avalanche-style — under controlled network delay and adversarial conditions, using a single discrete-event simulator. The contribution is not a benchmark of production systems but a reproducible cross-family evaluation framework in which the three are measured under matched assumptions. Performance is measured as commit latency, sustained throughput, and communication overhead. Security is measured as safety and liveness under adversarial pressure. The unified metric schema is defined in Chapter 3. The approach extends the simulation-based, metrics-instrumented methodology of Gervais *et al.* [14], originally applied to Proof-of-Work, to the three implemented BFT families. The comparison is organized around the Pareto frontier each family traces under matched conditions, and the question of whether any family dominates across all operating regimes is answered from experimental data.

1.4 Scope and assumptions

Evaluation is conducted at the message-passing level inside a discrete-event simulator. In scope are configurable network delay (constant, uniform, normal, exponential, heavy-tailed), configurable packet loss, three Byzantine validator behaviors drawn from the primary literature — silent non-participation, delayed voting, and equivocation — and validator sets of $n \in \{4, 7, 10, 16, 25\}$ nodes. Extrapolation to the several-hundred-node production scale rests on the sensitivity sweeps rather than on direct measurement at that scale. Out of scope are Proof-of-Work as a subject of comparison (it appears only as the methodological precedent [14]),

Layer-2 protocols, deployment on testnet or mainnet, economic and incentive design, and the performance of cryptographic primitives.

Simulation is chosen over testnet or live-network measurement for three reasons: reproducibility of seeded runs, controlled isolation of one independent variable at a time, and a matched harness across the three implemented families. The choice carries four framing assumptions. First, each family is represented by a deliberately simplified implementation, since the aim is fair like-for-like comparison rather than reproduction of any production codebase’s throughput. Second, the network is idealized: delay and loss are configurable parameters, but TCP congestion control, kernel scheduling, and physical-layer jitter are not modeled. Third, the adversarial strategies evaluated are those most frequently discussed in the primary literature; attacks requiring specialized cryptographic or economic modeling are left to future work. Fourth, published production figures are treated as order-of-magnitude sanity checks rather than validation targets; the simulator’s contribution is internal consistency across families under matched assumptions, not the reproduction of production throughput.

1.5 Research questions

Five research questions structure the evaluation. RQ1–RQ4 generate the data; RQ5 synthesizes it.

- **RQ1.** How does end-to-end commit latency scale, for each family, as the variance of the network-delay distribution increases from nominal to heavy-tailed? This tests the synchrony assumption each family makes.
- **RQ2.** How does sustained throughput degrade, for each family, as the Byzantine fraction approaches the theoretical fault threshold from below? This describes how each family approaches its fault bound, not only whether it reaches it.
- **RQ3.** What is the relative communication overhead of each family, measured in messages and bytes per agreed unit, under a fixed workload and identical network assumptions? This quantifies the asymptotic scaling each family claims.
- **RQ4.** Under which adversarial strategies — silent non-participation, delayed voting, equivocation — does each family show liveness degradation, safety violation, or neither? This maps each adversary onto the property each family claims to preserve.
- **RQ5.** Does a consistent Pareto frontier of the performance–security tradeoff exist across the three families evaluated, and does any family dominate across all operating regimes? This is the comparative synthesis and the primary contribution.

Each question is paired with a defined subset of the metric schema and a defined independent variable in the experiment matrix.

1.6 Contributions and thesis roadmap

The thesis makes four contributions:

1. **Artifact.** A discrete-event simulator for L1 consensus evaluation under configurable network delay and adversarial conditions, with a shared metric schema and a pluggable protocol interface.
2. **Implementations.** Simplified reference implementations of one representative protocol from each of the three families within a single harness — PBFT-style, PoS-finality, and Avalanche-style — enabling reproducible like-for-like comparison.
3. **Dataset and analysis.** An experimental dataset and comparative analysis quantifying the performance–security tradeoff across the three implemented families under matched conditions, answering RQ1–RQ4 and underwriting the RQ5 synthesis over those three.
4. **Methodological framing.** The simulation-based, metrics-instrumented approach of Gervais *et al.* [14], extended from Proof-of-Work to the three implemented BFT families on a single harness with matched assumptions.

Chapter 2 reviews the three families and the prior comparative evaluations that motivate a unified harness. Chapter 3 describes the methodology: the system model, the three protocol implementations, the metric schema, and the experimental design. Chapter 4 presents the baseline, network-delay, and adversarial results that answer RQ1–RQ4. Chapter 5 presents the cross-family Pareto synthesis that answers RQ5. Chapter 6 concludes with the findings, their limitations, and directions for further work.

Chapter 2

Literature Review

2.1 Blockchains and the consensus problem

A Layer-1 blockchain is a replicated, append-only log of transactions maintained by mutually distrusting validators. At each height every honest validator must commit the same block, and the mechanism that produces that agreement is the consensus protocol. Block agreement is, at each height, an instance of the Byzantine Generals Problem [1]: a set of processes, some of which may behave arbitrarily, must converge on a single value despite contradictory messages.

Three foundational results bound any such protocol. Lamport, Shostak and Pease [1] proved that deterministic agreement requires at least $3f + 1$ participants for f arbitrarily-faulty processes — the origin of the two-thirds supermajority recurring throughout the BFT literature. Fischer, Lynch and Paterson [2] showed that in a purely asynchronous network no deterministic protocol guarantees both safety and liveness against even one crash fault, which forces every protocol to relax some assumption. Dwork, Lynch and Stockmeyer [3] supply the most influential relaxation: under partial synchrony — delays are bounded, but the bound is not known a priori — consensus is solvable for $f < n/3$.

Every Layer-1 protocol therefore concedes along the synchrony axis, the fault-model axis, or both. The synchrony axis runs from synchronous through partial-synchronous and asynchronous to probabilistic. The fault-model axis runs from crash through omission to Byzantine, with stake-weighting where applicable. Wherever a family preserves deterministic safety, the $3f + 1$ quorum-intersection arithmetic recurs. The CAP theorem [15] makes the operational consequence concrete. Under partition a chain must choose Consistency or Availability, with partition-tolerance non-negotiable: PBFT-style and PoS-finality halt safely to keep Consistency, while Avalanche-style continues with weakened guarantees to keep Availability. The three properties expected of any consensus protocol — Agreement, Validity, and Termination — sit on top of these concessions and trade off against one another whenever the assumed model is violated.

2.2 The fork in the road

No foundational result permits a free lunch: every protocol begins by choosing which constraint to relax. PBFT [16] and its descendants HotStuff [17] and Tendermint [8] take the most direct point — a $3f + 1$ quorum under partial synchrony. The other two families are best read as answers to a single question: *once PBFT's point is fixed, which assumption is loosened next?* Figure 2.1 traces the propagation from the Byzantine Generals Problem to the three families.

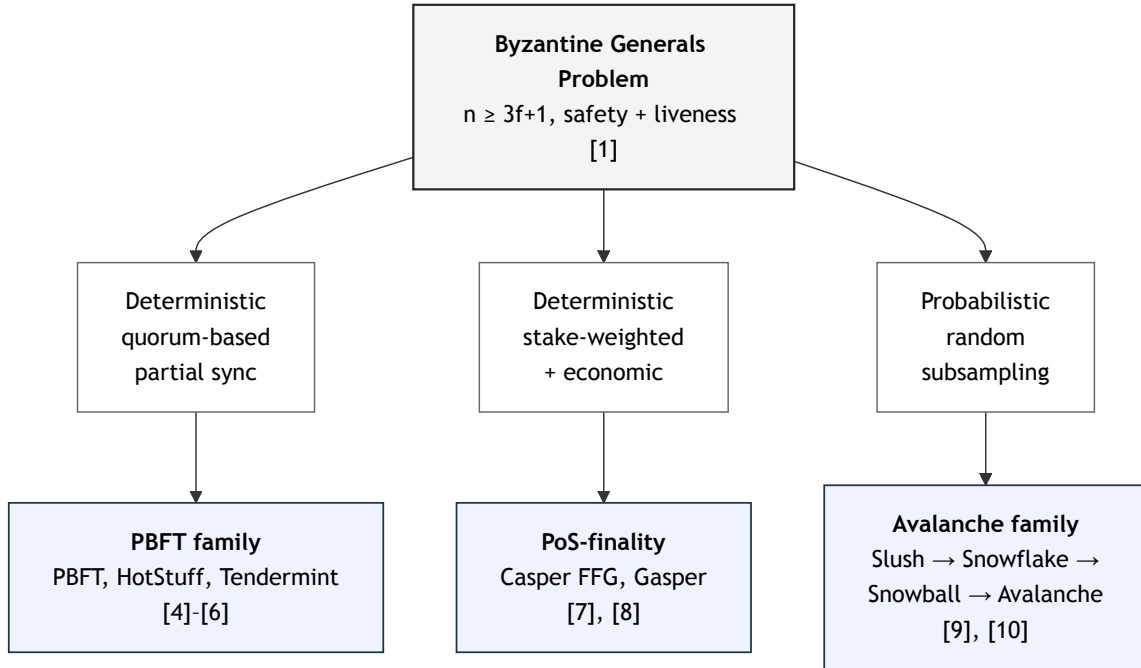


Figure 2.1: From the Byzantine Generals Problem to the three families.

- **PoS-finality** keeps the $3f + 1$ quorum and partial synchrony but adds an *economic* layer: a validator that equivocates can be slashed, converting the Byzantine fault model from an external assumption into a disincentivized behavior [5,18].
- **Avalanche-style** abandons the quorum entirely, replacing it with repeated random sampling and accepting a non-zero safety-violation probability ε in exchange for asynchrony tolerance and per-validator cost independent of n [11,19].

The choice each family makes determines the vocabulary in which it later reports performance (§2.4).

2.3 The three families

Table 2.1 traces the same event — committing one proposed block — through all three families. They do one thing in common: accumulate *agreement* on a block until reversal is impossible. They differ on three axes: what counts as agreement, how much of it locks

a block, and how many times that must happen. Read down those three rows: the two deterministic families are nearly identical: about two-thirds agreement, met twice, giving $\varepsilon = 0$. Avalanche-style breaks the pattern: about 80% of a small random sample, repeated β times, giving a small positive ε .

Table 2.1: **How one block becomes irreversible.** Protocol-native terms are in parentheses; notation is defined below the table; per-family citations appear in §2.3.1–§2.3.4.

	PBFT-style	PoS-finality	Avalanche-style
Synchrony assumption	partial synchrony	partial synchrony	asynchronous / probabilistic
Block proposer	one rotating leader (primary)	one leader per slot (stake-weighted)	one leader per slot (round-robin)
Unit of agreement	a replica’s vote (PREPARE/COMMIT)	an attestation, weighted by stake	a polled peer’s preference (QUERY-RESPONSE)
Agreement threshold	$\sim 2/3$ of all validators ($2f+1$)	$\sim 2/3$ of all stake	$\sim 80\%$ of a small random sample (α_c of K)
Agreement rounds	twice (prepare, commit)	twice, ≥ 2 epochs apart (justify, finalize)	$\beta \approx 15$ sampling rounds in a row
Finality guarantee	deterministic, $\varepsilon = 0$	deterministic, $\varepsilon = 0$	probabilistic, $\varepsilon \leq (1 - \alpha_c/K)^\beta$
Communication cost	$O(n^2)$	$O(n)$, BLS-aggregated to ~ 1	$O(K \cdot \beta)$, independent of n
Adversarial pressure point	stall the leader (liveness); safety holds $< n/3$	stall finality; breaking safety burns $\geq 1/3$ stake	stall the counter (liveness); safety statistical

Notation.

n, f validators and the maximum Byzantine count; $n = 3f + 1$ for the quorum-based families.

K peers polled per round (Snowman); $K \approx 20$, or $\min(20, n-1)$ at thesis scale.

α_c sampled peers that must agree to count a round; $\alpha_c \approx 0.8K$, and holding $\alpha_c/K \approx 0.8$ fixes the safety-curve shape.

β consecutive agreeing rounds before a block is accepted; $\beta \approx 15$, and holding β fixed makes ε invariant in n .

ε probability two honest validators commit conflicting blocks; 0 for the two deterministic families, $\leq (1 - \alpha_c/K)^\beta$ for Snowman.

At $n = 7$, the two deterministic families lock a block with a ~ 5 -of-7 agreement (or $2/3$ of stake), met twice. Snowman instead needs ~ 5 -of-6 polled peers to agree 15 rounds running — reaching $\varepsilon \approx 10^{-12}$ by repetition, not by one counted quorum.

The three paragraphs below add what the table cannot: the documented variants of each family and the adversarial weakness documented for it, within the §1.4 taxonomy (silent

non-participation, delayed voting, equivocation). The simulator-level treatment of each protocol is deferred to Chapter 3.

2.3.1 PBFT-style

Beyond classical PBFT [16], HotStuff [17] linearizes leader replacement to $O(n)$ with threshold signatures and Tendermint [8] rotates the leader round-robin. The documented weakness within the §1.4 taxonomy is leader-targeted delay: a Byzantine or delaying leader triggers successive view changes whose $O(n^3)$ cost in the original formulation [16] degrades latency well before any safety threshold is approached. Safety stays unconditional below $f < n/3$ — an equivocating leader is caught at the prepare round, where no two-thirds agreement forms for either value.

2.3.2 PoS-finality

Gasper [5] composes Casper FFG with the LMD-GHOST fork-choice rule and is the deployed Ethereum specification. The family’s signature property is accountable safety: two conflicting finalized checkpoints imply that at least one-third of stake signed a slashable message, making a violation attributable, not merely infeasible [18]. The most-studied weakness is equivocation. Slashing exists to deter it, and the safety proof depends on that slashing being credibly enforced [5,18]. The secondary weakness is prolonged silent non-participation, which leaks absent validators’ stake to restore a quorum while the time-to-finality window widens.

2.3.3 Avalanche-style

The production form is Snowman, the linearized variant of the Slush $\rightarrow$ Snowflake $\rightarrow$ Snowball $\rightarrow$ Avalanche cascade [11]. The family has no fixed $f < n/3$ threshold: the tolerated Byzantine fraction is set by the parameter choice. Its per-validator cost $O(K \cdot \beta)$ is independent of n . The documented weakness is equivocation. Amores-Sesar, Cachin and Schneider [19] show that an adversary influencing as few as two undecided validators can stall the confidence counter beyond any number of rounds polynomial in β , a far worse cost than the original analysis [11] implies. The same work confirms fast convergence absent such an adversary and proposes a fix.

2.4 Why the published numbers do not compare

The three families have all been measured, and the field has produced three taxonomic surveys plus one quantitative methodological precedent. None yields a cross-family comparison under matched conditions, for two distinct reasons.

2.4.1 Each family reports in its own vocabulary

Each family’s primary papers report what its design concedes, and little else. The PBFT and HotStuff papers report operations per second on a low-latency LAN with view-change

cost as the disturbance metric [16,17]. Casper FFG reports time-to-finality in *epochs*, a unit that becomes physical only after multiplication by a slot duration [5,18]. Avalanche reports two figures absent elsewhere: the probabilistic safety bound ε and a per-transaction latency under one K, α, β parameterization [11,19]. Table 2.2 collects the headline numbers.

Table 2.2: **Reported metric vocabulary across families.** Headline numbers from the cited primary sources. The columns are not directly comparable.

Family	Throughput (reported)	Latency (reported)	Fault threshold
PBFT (LAN) [16]	Thousands of ops/s	Sub-10 ms	$< n/3$
HotStuff [17]	Linear in n after optimizations	3-round commit	$< n/3$
PoS-finality [5,18]	Block-proposal rate of underlying chain	Two-epoch finality (≤ 12.8 min on Ethereum)	$< 1/3$ stake
Avalanche [11,19]	~ 3.4 ktps (testnet)	~ 1.35 s	Parameter-dependent

Hardware, workload, batching, topology, and adversarial assumption differ between every pair of rows. The table cannot answer whether PBFT’s thousands of ops/s on a LAN reflects a family advantage or a more permissive harness. Nor can it answer whether Avalanche’s 1.35 s confirmation is structural or an artifact of one K, β choice. The comparative question is hard to phrase before it is hard to answer.

2.4.2 The surveys measure nothing, and the one precedent targets PoW

Three taxonomic surveys place the families in qualitative terms. Bano *et al.* [12] supply the canonical Systematization of Knowledge and the taxonomic backbone this chapter reuses. Xiao *et al.* [20] aggregate reported throughput, latency, and fault-tolerance ranges across families while flagging the cross-harness incomparability. Cachin and Vukolić [13] supply a methodological critique of permissioned-chain BFT that questions the robustness of vendor-published numbers. None measures the families; the two that aggregate numbers do so under the caveat that the harnesses are not matched.

The single quantitative precedent is Gervais *et al.* [14]. Their unified Proof-of-Work simulator — one metric schema, swept over block size and propagation delay, applied uniformly to the protocols compared — is structurally the evaluation the BFT families have not received. Its limitation, for this thesis, is its target: Proof-of-Work, not the three BFT families. No equivalent unified-harness study exists for PBFT-style, PoS-finality, and Avalanche-style protocols evaluated jointly.

2.5 The gap

The three families respond to the same impossibility and differ only in which assumption they relax, so they are commensurable as objects of comparison. Their per-family measurements, however abundant, do not support cross-family judgment. The obstacle is the reporting

vocabulary itself, not any absence of measurement. The one methodological precedent for a matched evaluation is Gervais *et al.* [14], which has been applied to Proof-of-Work only. Chapter 3 responds with a unified discrete-event simulator, a shared metric schema instantiated identically for the three families, and an experiment matrix that sweeps the network-delay, Byzantine-fraction, validator-set-size, and adversarial-strategy axes under common assumptions.

Chapter 3

Methodology

3.1 Overview

This chapter describes the simulator that closes the gap of Chapter 2 and operationalizes the four data-generating research questions, RQ1–RQ4: a single discrete-event system in which the three families run under one system model (§3.2), one metric schema (§3.5), and one experiment matrix (§3.4) that varies each research question’s independent variable in isolation. The fifth question, RQ5 — whether a consistent performance–security Pareto frontier emerges across the families — is a synthesis over the RQ1–RQ4 data rather than a sweep this matrix prescribes, and is answered in Chapter 5. The approach extends the instrumented-harness methodology of Gervais *et al.* [14] from Proof-of-Work to the three BFT families.

Three protocols are implemented: PBFT, Casper FFG, and Snowman. One system model (§3.2), one simulation setup (§3.4), and one metric schema (§3.5) apply uniformly across them.

3.2 System model

The three consensus families decide in genuinely different ways, and that diversity is the comparison problem this chapter must solve. They are not one protocol run with different constants. They differ in leadership, in what one decision commits, and in layering (established in §2.3, catalogued per protocol in §3.3, Table 3.3). The three do not even produce the same *kind* of decision, so a fair comparison cannot be read off raw numbers. It needs two things: a single fixed engine that runs all three identically, and a downstream step that reconciles their differing output onto one scale.

The spine of that engine is short (Figure 3.1). One seed and one configuration enter; the harness builds the runtime machinery — scheduler, network, validators, logger — identically for every protocol, swapping only the protocol-logic slot; a single run loop (detailed below) drives the run; and a reducer turns the recorded event stream into one comparable row. The same machinery runs once per seed and once per cell of the experiment matrix (§3.4), so the only thing that varies between two rows is the quantity under study.

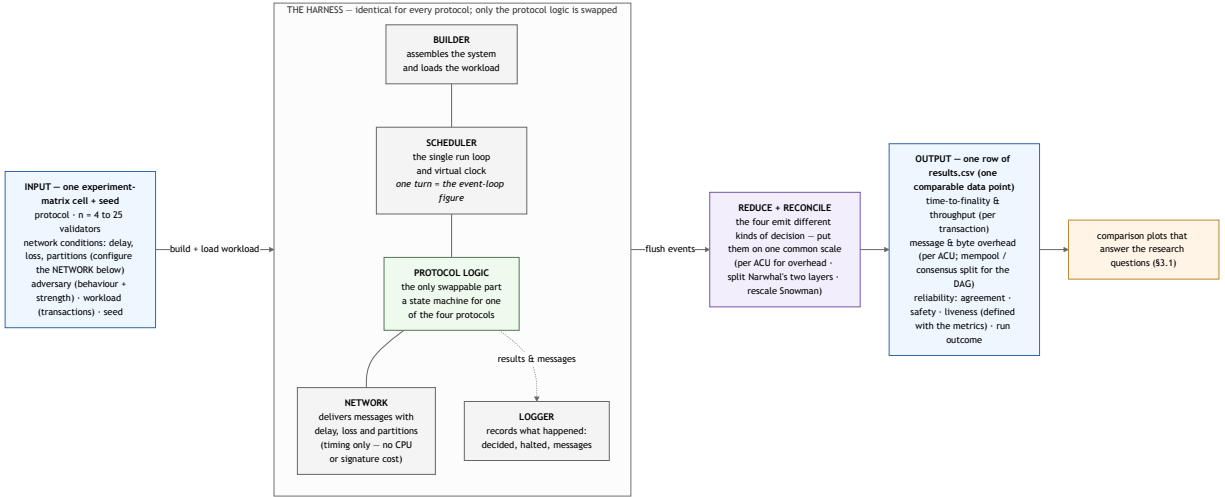


Figure 3.1: Structural view: a single fixed harness in which only the protocol-logic slot is swapped, turning one experiment-matrix cell and seed into one comparable per-trial row.

A run is fully described by the configuration that enters it. The configuration loader requires seven top-level keys, shown here as the input contract:

```

n: 10                                # validator-set size
t_max: 20.0                          # virtual-time deadline
seeds:
  n_runs: 20                        # the harness enumerates seeds 0 ... n_runs-1
network:
  phases:                            # a TIME-STAMPED sequence of phases, not a static setting
    - t_start: 0.0
      t_end: 20.0
      delay: { kind: "...", params: { ... } }
      p_drop: 0.0                    # optional --- per-phase message-loss probability
      partitions: []                 # optional --- disjoint validator groups
  adversary: { ... }                 # opaque block --- interpreted per protocol/adversary
  protocol_knobs: { ... }            # opaque block
  workload: { ... }                  # opaque block
  
```

Two features of this contract carry weight later. The **network** key is not a static setting but a time-stamped sequence of phases, each fixing a delay regime and optional message loss and partition over a stated interval. Scheduling the regime change in advance is what lets one seeded run cross a partial-synchronous Global Stabilization Time without mid-run intervention. The **adversary**, **protocol_knobs**, and **workload** blocks are deliberately opaque. The loader admits each whole and leaves interpretation to the protocol or adversary that consumes it, which is how one harness admits three protocols that need not share a knob schema.

When wired, each validator is granted exactly three capabilities, the only ways it affects anything outside itself. Each is owned by a different component:

Table 3.1: **Validator capabilities and their owners.**

Capability granted to a validator	Owned by	Purpose
set / cancel a timer	scheduler	a validator schedules its own future wake-ups
send / broadcast a message	network	a validator communicates with other validators
emit an event (decided , halted , ...)	logger	a validator records what happened

This three-capability contract is the split-ownership invariant: the scheduler owns time and event delivery, the network owns transport, the logger owns the record. Because the builder, scheduler, network, and logger are identical across protocols, any difference between two rows is attributable to the protocol logic alone. The network models timing only: it delivers each message at most once, unordered, under per-phase delay, loss, and partition. Every protocol’s messages travel in one uniform envelope distinguished only by a per-protocol type (Table 3.3).

The engine’s mechanism is one run loop (Figure 3.2) — the scheduler box of Figure 3.1 seen up close. One turn proceeds as follows. If the virtual clock has reached the deadline `t_max`, the run stops. If no events remain, the run stops on quiescence. Otherwise the scheduler pops the soonest event, identified by the triple `(t, node, seq)`, and advances the virtual clock to `t`. If that event is a timer that has since been cancelled, it is skipped. Otherwise the logger records it and the scheduler hands it to the target validator, which reacts and may schedule new events; the loop then checks the caller’s stop predicate before taking the next turn. A run therefore ends on one of three termination paths — **quiescence** (the heap is empty), **deadline** (the clock reached `t_max`), or **predicate** (a caller-supplied condition became true). The loop is identical for all three protocols; only the state machine inside the validator differs.

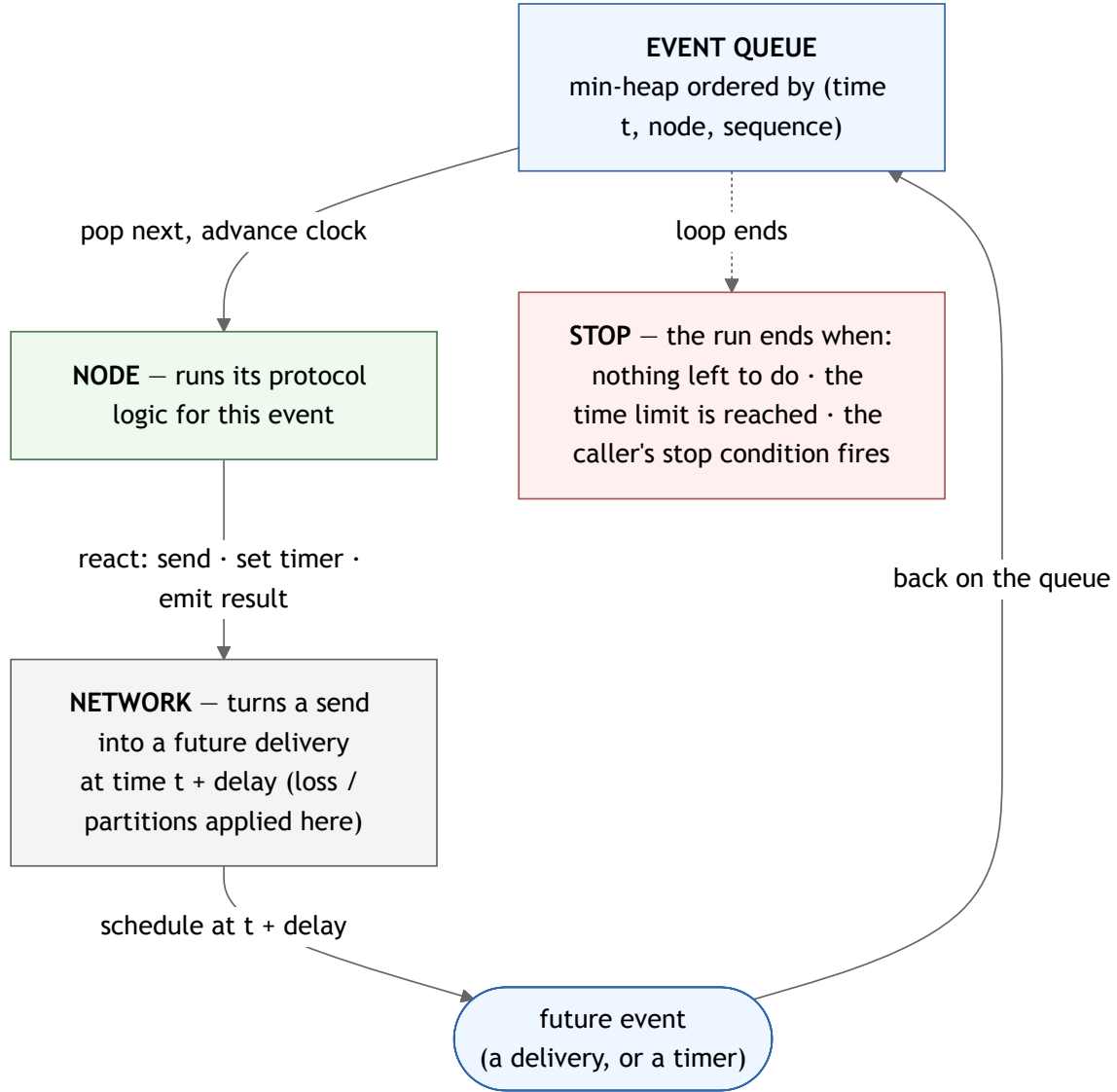


Figure 3.2: One turn of the scheduler’s event loop — pop the next event, advance the virtual clock to it, let a node react, enqueue what it produces — with the three termination paths.

Two properties of this model are load-bearing. First, determinism: the event queue breaks ties among same-time events by a fixed total order, and every random draw derives from a single global seed keyed by stream identity rather than by construction order. A configuration paired with a seed therefore reproduces a byte-identical event stream, so any reported number is exactly reproducible. Second, network phases are scheduled in advance, so a single seeded run can cross a partial-synchronous Global Stabilization Time without intervention.

Isolation is not commensurability. Running three kinds of decision through one identical engine secures only that the engine adds no difference of its own. Making them comparable is a separate downstream step: the reconciliation step the metric schema performs under stated conventions when the per-trial rows are read back (§3.5). Finally, an adversary is not a new component but a per-node interceptor that alters a single validator’s outgoing

messages; the Byzantine strategies a profile can apply are catalogued in the adversary model, and §3.4 names the three the RQ4 sweep exercises.

3.3 Algorithms

Each family surveyed in Chapter 2 (§2.3) is represented in the simulator by a single protocol; Table 3.2 fixes that correspondence, and its rightmost column states why each protocol is a faithful representative of its family. The family labels are those of Chapter 2; the protocol names are used from here onward.

Table 3.2: **Family-to-protocol mapping.**

Family (Chapter 2)	Implemented protocol	Why it represents the family
PBFT-style	PBFT	The canonical leader-driven, multi-phase BFT protocol the family is named for; its $3f + 1$ quorum and view-change are the mechanism every later variant refines.
PoS-finality	Casper FFG	The finality gadget Ethereum deploys; it carries the family’s defining stake-weighted, two-checkpoint justification rule.
Avalanche-style	Snowman	The production linear-chain form of the Avalanche family; it exercises the family’s defining repeated random-subsample voting.

Chapter 2 (§2.3, Table 2.1) establishes the *mechanism* of each protocol family; this section does not revisit it. It audits where the simulator’s implementation *departs* from the textbook family, and why each departure leaves the comparative results valid. Table 3.3 summarizes the three protocols at a glance — whether each elects a leader, what one decision commits (its *atomic commit unit*, or ACU; defined in §3.5), the message types each carries, the event that signals a commit, the knobs exposed to experiments, and the main simplification each makes. Each subsection then pairs the protocol’s decide-path figure — the honest path from first message to **decided** — with a *deviation ledger* whose numbered entries name every point at which the implementation departs from the family and the validity boundary each departure introduces.

In this table and the subsections that follow, n is the validator-set size and f the Byzantine-fault threshold it tolerates ($n = 3f + 1$, §3.4.2); the Snowman tuple $(K, \alpha_p, \alpha_c, \beta)$ is defined where it is set, in §3.3.3.

Table 3.3: **The implemented protocols in the simulator.**

	PBFT	Casper FFG	Snowman
Leader?	Yes — single rotating primary	Yes — slot proposer	No — leaderless
Decision unit (ACU)	one committed block	one finalized checkpoint (+ its ancestors)	one accepted block
Message types	PRE-PREPARE, COMMIT, NEW-VIEW, PREPARE, VIEW-CHANGE	BLOCK-PROPOSAL, ATTESTATION, SLASHING-EVIDENCE	BLOCK-ANNOUNCEMENT, QUERY, QUERY-RESPONSE
decided fires when	$2f+1$ COMMIT collected for one (view , seq)	a checkpoint and its direct child are both justified	the confidence counter reaches β
Knobs exposed	view-change timeout ($3 \cdot E[\text{round latency}] \cdot 2^{\text{view}}$ backoff); Byzantine fraction	<code>slots_per_epoch</code> (4); <code>slot_duration</code> (100 ms); justification threshold	$(K, \alpha_p, \alpha_c, \beta)$ via the rescaling rule; sampling seed
Main simplification	classical variant only — no HotStuff/Tendermint, no signatures	no LMD-GHOST fork choice; attest once per epoch; slashing modeled as a halt	linearized Snowman only — no DAG-Avalanche; no stake-weighted sampling

3.3.1 PBFT

Only the classical variant is implemented. Figure 3.3 traces its honest three-phase commit and the view-change branch; the deviation ledger records where the implementation departs from classical PBFT.



Figure 3.3: PBFT three-phase commit for one (view, seq) instance with the view-change branch.

Deviation ledger.

- ① **Single primary — classical variant only.** No HotStuff threshold-signature linearization and no Tendermint rotation.
- ② **Exponential view-change backoff.** The view-change timeout uses a per-view backoff $vc_delay \cdot 2^{\text{view}}$, required so recovery terminates deterministically: a flat timeout that has fired once fires again in the same delay regime.
- ③ **Digests, not certificates.** The simulator carries no cryptographic signatures; message digests serve integrity and deterministic replay only, so a `VIEW-CHANGE` message's prepared evidence is an assertion rather than a cryptographic certificate, and the adversary catalog deliberately carries no evidence-forgery capability. *Validity boundary:* this affects no honest-path result and no equivocating-primary result; within-threshold safety against forged view-change evidence is assumed by construction.

- ④ **Worst-case message complexity.** Classical PBFT sits at the $O(n) \rightarrow O(n^2)$ end of the family’s message-complexity range, so the RQ3 verdict reported for “PBFT-style” is a verdict on classical PBFT specifically.

3.3.2 Casper FFG

The implementation is a simplified Casper FFG gadget. Figure 3.4 traces one epoch’s justify→finalize path and its slashing branch; the deviation ledger records the departures from deployed Ethereum FFG.



Figure 3.4: Casper FFG justify→finalize for one epoch with the slashing branch.

Deviation ledger.

- ① **Deterministic stake-weighted proposer.** Proposer assignment is a pure function of (global_seed, slot) through blake2b, verified for fairness at four stake distributions.
- ② slots_per_epoch = 4 (against Ethereum’s 32) — the smallest value that still admits a multi-slot epoch, which preserves FFG’s epoch character.
- ③ slot_duration = 100 ms (against 12s) — keeps FFG finality at the same order of magnitude as the other protocols’ commit latency, so cross-protocol plots remain readable on one axis. *Defence:* a sensitivity sweep at {16, 32} and {500 ms} tests whether the comparative ordering is preserved at production scale (results in Chapter 4).

- ④ **No LMD-GHOST fork choice.** `Chain.head` is the block at the greatest known slot under an honest-path linear chain.
- ⑤ **Slashing modeled as a halt, detection only.** A provable double- or surround-vote halts the run with the offending validators recorded; no deposit is destroyed and no stake is removed from the active set, so the economic penalty — and the safety-cost budget it would yield — lies outside scope, consistent with the economic-design exclusion of §1.4.

3.3.3 Snowman

Only the linearized Snowman variant is implemented; full DAG-Avalanche is out of scope, which keeps the chain structure directly comparable to PBFT and Casper FFG under the shared `Node` interface. Production Snowman runs on validator sets in the thousands with $(K, \alpha_p, \alpha_c, \beta) = (20, 11, 16, 15)$ — respectively the poll sample size, the preference and confidence thresholds, and the decision threshold — so the thesis-scale sweep $n \in \{4, 7, 10, 16, 25\}$ requires the rescaling rule the deviation ledger records. Figure 3.5 traces the poll loop.

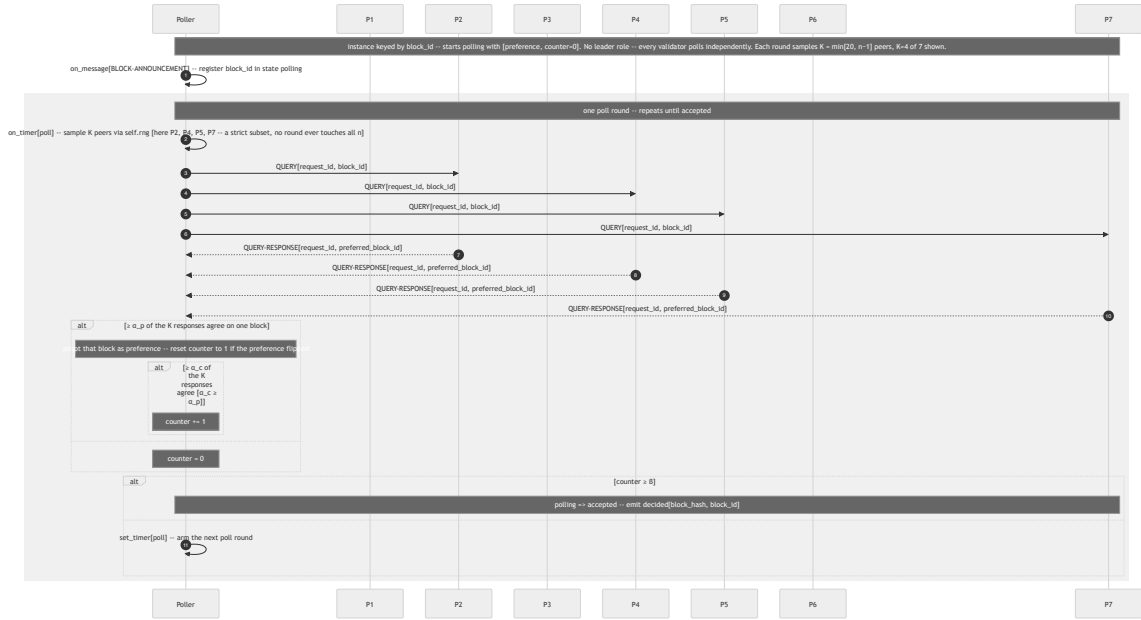


Figure 3.5: Snowman subsampled K -peer poll loop for one block, accepting at counter $\geq \beta$.

Deviation ledger.

- ① **Round-robin proposer.** Proposer assignment is $\text{slot} \bmod n$; Snowman is not stake-weighted.
- ② **Sample size rescaled.** $K = \min(20, n-1)$, $\alpha_p = \lfloor K/2 \rfloor + 1$, so the poll fits a thesis-scale validator set.

- **③ Confidence threshold rescaled, shape held.** $\alpha_c = \lceil 0.8 \cdot K \rceil$. Snowman’s safety bound is $\varepsilon \leq (1 - \alpha_c/K)^\beta$ — the analytical ceiling on the probability ε that two honest validators accept conflicting blocks, exponentially small in the confirmation depth β . Holding the ratio $\alpha_c/K \approx 0.8$ across the sweep preserves the *shape* of that bound rather than its numerical value, because the exponential form is what carries the probabilistic-finality semantics. The ceiling rounds α_c up, so the realized ratio never falls below 0.8 and the rescaled bound is at least as tight as production.
- **④ $\beta = 15$ held fixed.** Holding β fixed keeps the *exponent* of the safety bound $(1 - \alpha_c/K)^\beta$ constant across the sweep, preserving the exponential form that carries the probabilistic-finality semantics. It does not hold the *realized* bound constant. The base $(1 - \alpha_c/K)$ tracks the ceiling-rounded ratio of ③, so the analytical ceiling ε varies non-monotonically with n , from $\approx 10^{-11}$ at $n \in \{16, 25\}$ down to $\approx 10^{-15}$ at $n = 10$, several orders of magnitude. The smallest sets sit nearest unanimity ($n = 7$ gives $\alpha_c/K = 5/6 \approx 0.833$). Together with ②–③, the rescaled protocol is therefore one Snowman in *form* across the sweep — the same exponential semantics tracked at fixed β — rather than an identical numerical guarantee at each n .

Degeneracy (excluded). The rescaling degenerates at $n = 4$, where it yields $\alpha_c = K = 3$: every poll then queries all peers and demands unanimity, collapsing Snowman to flood-voting and driving the bound $(1 - \alpha_c/K)^\beta$ to zero. This is not because finality is genuinely perfect, but because this parametrization is no longer Snowman, only the degenerate boundary of the rescaling rule. The $n = 4$ Snowman row is therefore excluded from the comparative tables of Chapters 4 and 5 and reported once as a rescaling sanity check, leaving PBFT and Casper FFG at $n = 4$ unaffected.

Honest-path check. The honest path is verified at $n \in \{4, 7, 10\}$ by the Snowman baseline experiment, in which every validator decides every announced block with no forks and byte-identical replay.

3.4 Simulation setup

3.4.1 Reproducibility and the run lifecycle

The harness wires and runs the system by the bootstrap and run loop of §3.2, and the same configuration paired with the same global seed produces a byte-identical event stream, since the bootstrap adds no randomness of its own. Two points specific to these experiments follow.

First, the baseline experiments do not load a YAML file per run: they build the configuration programmatically, as scenario definitions in code that produce the same `Config` object the loader would, so the seven-key schema of §3.2 is the documented contract rather than an on-disk artifact for every cell.

Second, the three termination predicates of §3.2 (*quiescence*, *deadline*, *predicate*) interact with the measurement window: time-bounded runs use a buffer beyond the window, and the analysis step clips out-of-window events. A *deadline* stop is therefore not in itself a liveness failure — a run is scored as a liveness failure only when no honest validator commits

inside the window, the condition the `success_rate` column reports against `t_max`. This keeps the RQ4 success-rate column free of runs that were healthy but truncated.

3.4.2 The experiment matrix

An experiment is one point in the product of six axes — validator-set size n , network timeline, adversary, protocol knobs, workload, and seed. The sweep $n \in \{4, 7, 10, 16, 25\}$ is $3f + 1$ at $f \in \{1, 2, 3, 5, 8\}$, giving a clean Byzantine-threshold instance at each point. Two fault symbols recur and are kept distinct throughout: f is the integer fault threshold a configuration tolerates ($n = 3f + 1$), while φ is the adversarial fraction actually injected in Family C — a real fraction of the validator set, independent of the $n = 3f + 1$ relation. Experiments group into three run families, each fixing five axes and sweeping one:

- **Family A (Scaling)** sweeps n under an honest validator set on the `static-baseline` network, for RQ3.
- **Family B (Delay)** sweeps the network timeline at $n \in \{10, 25\}$, for RQ1.
- **Family C (Adversarial)** sweeps the adversary at $n \in \{10, 25\}$, for RQ2 and RQ4 jointly; the capability set, intensity grid, and magnitude axis are detailed below.

$n = 10$ is the shared anchor for Families B and C — the middle of the scaling sweep, $3f + 1$ at $f = 3$, small enough to afford many seeds — and both families also run at $n = 25$ ($3f + 1$ at $f = 8$) to amplify the delay and adversarial effects.

The network timeline Family B sweeps is built from the per-phase delay model of §3.2. Each phase fixes a delay distribution from a fixed catalogue — `constant`, `uniform`, `normal`, `exponential`, and `heavy_tail` (Pareto) — plus an optional message-loss probability and an optional partition. The sweep moves from the `static-baseline` network (one `constant` phase over $[0, t_{\max})$) through a moderate regime (`uniform`, 100–500 ms) to a heavy-tailed regime (1–5 s with a long tail); partial synchrony is the two-phase case — an asynchronous, heavy-tailed phase, optionally partitioned, before the Global Stabilization Time, then a bounded-delay phase after it — so one seeded run crosses GST without intervention.

Family C fixes the network at `static-baseline` (a constant 10 ms delay) and sweeps the adversary — the per-node interceptor of §3.2 — the mirror of Family B. An `AdversaryProfile` is static data: its capability, intensity, and bound node set are fixed at sim-start and never adapt mid-run. Three capabilities are exercised, one per Byzantine behavior of RQ4, each at an intensity given by the adversarial fraction φ — denominated in each protocol’s natural unit (replicas for PBFT, validators for Snowman, stake for Casper FFG) and swept over a sub-threshold band $\varphi \in \{0.10, 0.20, 0.30\}$ against a $\varphi = 0$ honest control:

- **delay-emission** (delayed voting) — hold an outbound vote past the protocol’s timing tolerance; adds a magnitude axis $m \in \{2, 4, 6, 8, 10\}$, the forced delay as a multiple of each protocol’s round cadence.
- **withhold-participation** (silent non-participation) — a silent validator that still runs its state machine but emits nothing, the crash-faulty case.

- **equivocate-vote** (equivocation) — sign two conflicting messages where the protocol expects one. This safety-relevant sweep also drives PBFT and Casper FFG above the $1/3$ bound ($\varphi \in \{0.40, 0.50\}$) to expose the safety cliff; Snowman cannot fork below threshold and is not swept above it. For Snowman the capability has no distinct realization and reduces to a “lying responder” that coincides in effect with **withhold-participation**.

Workload defaults are:

- a Poisson arrival process;
- fixed 512-byte transactions;
- a zero conflict rate;
- an offered rate of 100 transactions per second in the sub-saturation latency regime.

Because the network is latency-only (§3.2), the simulator has no saturation point — a block of one transaction and one of a thousand commit at the same simulated latency — so sustained throughput is measured as goodput at this fixed sub-saturation rate; a peak-throughput measurement is deferred to a task that first adds a capacity or cost model rather than reported as a configuration artifact.

The three protocols share the same seed set at every configuration point, and because randomness is keyed by stream identity (§3.2), all three draw from the same network and arrival randomness — so the cross-protocol comparison is paired under common random numbers, a variance-reduction technique on the paired differences. The seed count this pairing affords, and the interval machinery that reads it, are set out with the metric schema (§3.5).

3.4.3 Regime-coherence constraints

Two coherence constraints keep each protocol in its own regime while it sits on the shared axes. The FFG runner refuses any pairing of the baseline `slot_duration` = 100 ms (§3.3.2 ③) with a network phase whose $E[\text{delay}]$ — the mean delay of the phase’s distribution — is not far below the slot duration. When $E[\text{delay}]$ approaches `slot_duration`, attestations from distant validators arrive after the slot boundary, producing a degraded regime that is not Casper FFG. Family B therefore rescales `slot_duration` upward with the delay regime, by the rule `slot_duration` $\geq 4 \cdot E[\text{delay}]$. The matrix owns the per-timeline pairing, and the runner refuses to start an incoherent pairing rather than report degraded-finality numbers labeled as Casper FFG. Because the slot duration grows with the delay regime, Casper FFG’s time-to-finality in Family B is slot-dominated and necessarily rises with delay. The RQ1 curve for Casper FFG therefore reports the protocol’s *delay coupling*, the same coupling Ethereum’s 12s slot reflects, rather than a free-running delay-sensitivity measurement. It is read as such rather than hidden. The second constraint is the analogous one for Snowman: its (K, α_p, α_c) are not free axes but functions of n fixed by the §3.3.3 rescaling rule (which also fixes the $n = 4$ Snowman exclusion recorded there).

3.4.4 One run, end to end

To make the matrix concrete, consider one cell: PBFT at $n = 10$ (so $f = 3$ and the quorum is $2f + 1 = 7$), the **static-baseline** network, an honest validator set, Poisson arrivals at 100 tx/s, and seed 0. The six phases run as follows; Figure 3.6 gives the same run as a temporal sequence.

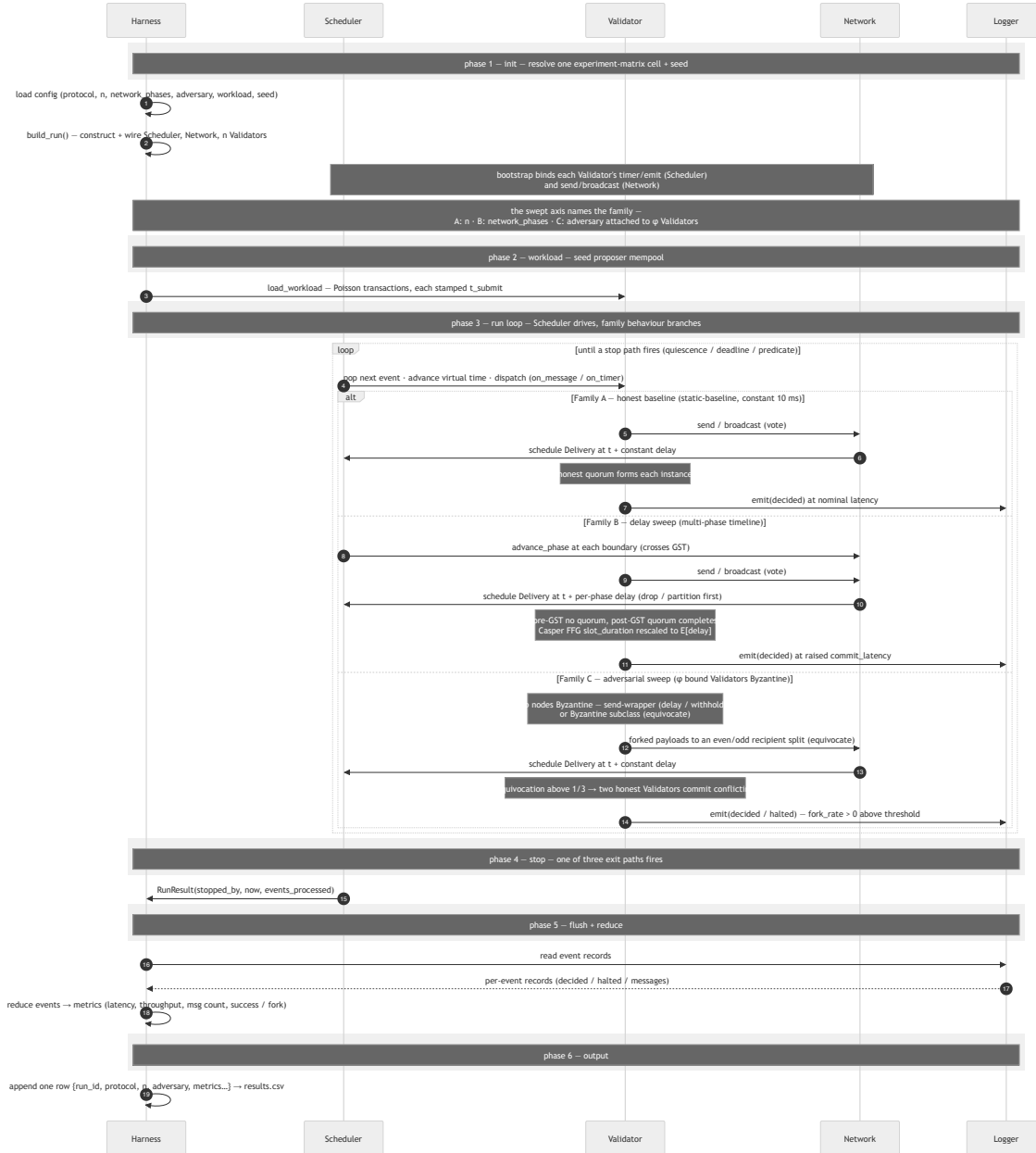


Figure 3.6: One seeded run as a temporal sequence of the six phases — init, workload, run loop, stop, flush, output — producing one **results.csv** row, with the run-loop branch showing where the delay (Family B) and adversarial (Family C) sweeps diverge from the honest baseline (Family A). The adversary is drawn inside the Validator lane, not as a separate component: it is the per-node interceptor of §3.2 that alters a bound validator’s outgoing messages.

1. **Init.** The harness builds and wires the scheduler, network, and ten validators, and names one of them the primary.
2. **Workload.** The harness seeds the primary’s mempool with transactions drawn from

the Poisson process; each transaction τ is stamped with its submit time t_{submit} .

3. **Run loop.** At its propose timer the primary batches τ into a block and broadcasts PRE-PREPARE. Each validator that accepts it broadcasts PREPARE; once a validator has collected 7 matching prepares it broadcasts COMMIT; once it has collected 7 matching commits it emits a **decided** event for the block at time t_{decided} . The network applies the **static-baseline** delay to every delivery, and the scheduler advances virtual time only as it pops events.
4. **Stop.** The run ends at **quiescence** or **t_max**.
5. **Flush and reduce.** The harness reduces the event stream to metrics. The commit latency of τ is $t_{\text{decided}} - t_{\text{submit}}$, and throughput is the committed-transaction count over the measurement window. **consensus_msgs_per_acu** is the per-ACU message cost: the PRE-PREPARE broadcast, the all-to-all PREPARE and COMMIT rounds, and the client REPLYs total $2(n^2 - 1)$ deliveries ($O(n^2)$ traffic), which over the n **decided** events is $(2n^2 - 2)/n = 2n - 2/n$ (≈ 19.8 at this n).
6. **Output.** One per-trial row, tagged with **(protocol, n, seed)**, is appended to the per-trial CSV.

The row this run appends has the shape of Table 3.4, shown with a few salient columns and the remainder elided.

Table 3.4: **Shape of one comparative-CSV row** (the **pbft-n10**, **seed-0** run). Salient columns are shown; “...” marks the elided remainder of the full 24-column schema fixed in §3.5.

run_id	protocol	n	seed	...	commit_hash	t_max	commit_latency_ms
pbft-n10	pbft	10	0	...	24a491a4	20.0	1000.000003...

The **commit_hash** (24a491a4) and **seed** columns embedded in every row are the hard evidence of reproducibility: together they pin the exact code and the exact random draws that produced the row, so any number can be regenerated from the record alone. The full 24-column schema is defined in §3.5 and is not repeated here.

The walkthrough above produces one row of a single file, but the comparison rests on two files. The first is a per-trial, long-format CSV — **results/baseline/baseline.csv** — that carries one row per **(protocol, scenario, seed)**; this is the file step 6 appends to, one row per run. The second is produced by a separate, downstream aggregation step: a wide CSV — **results/baseline/aggregated.csv** — that carries one row per configuration, holding the mean and 95% confidence interval of each metric across the seed set, and it is this aggregated file that feeds the Chapter 4 plots. The mean and confidence interval are therefore properties of the aggregation, not of any single per-trial row.

Repeating the run over seeds $0 \dots 19$ under common random numbers gives the aggregation step the sample it reduces to the cell’s mean and 95% confidence interval. Family B replaces the network phase, Family C attaches an **AdversaryProfile** to φ of the validators, and the

other two protocols substitute their own proposer, message types, and **decided** condition (Table 3.2); the six-phase lifecycle is identical for all three.

3.5 Metric schema

The three protocols do not emit commensurable events: PBFT commits a block, Casper FFG finalizes a checkpoint, and Snowman accepts a block once its counter reaches β . They differ structurally — PBFT’s and Snowman’s per-block finality against Casper FFG’s per-epoch finality, and Snowman’s parameter rescaling — so no quantity can be read off the raw event stream and compared across families until it is first placed on a common axis. Building that axis is the work of the metric schema, and the schema is uniform across families: each metric has one definition, one unit, and one fixed instrumentation point in **src/**, and the family-specific differences appear only as different per-protocol formulas computing the same column. Four metric families cover the evaluation:

- *latency* measures how quickly a transaction reaches commit or finality;
- *throughput* measures how many transactions the protocol commits per unit time;
- *overhead* measures the messages, bytes, and per-validator state it consumes;
- *reliability* measures whether it preserves safety and liveness under delay and adversary.

The device that makes the three commensurable is the *atomic commit unit* (ACU): the smallest contiguous set of transactions a protocol commits indivisibly. Every “per-block” metric is rewritten as “per ACU”, where the ACU is one block for PBFT, one finalized checkpoint for Casper FFG, and one block for Snowman — so one denominator serves all three without a conditional formula. The three protocols are thereby plottable on one scale under three stated conventions — the ACU denominator, the Snowman parameter rescaling, and the Casper FFG calibration of §3.3.2. Because each convention is a modeling choice rather than a neutral fact, a verdict is reported as robust only when it survives the sensitivity sweep that varies the convention’s governing knob.

Latency carries one further convention. Every latency metric is anchored at the transaction’s submit time, so end-to-end latency is comparable across protocols whether or not they carry a separate mempool. **commit_latency_ms** is the canonical cross-protocol time-to-finality axis: the simulator’s **decided** event fires at each protocol’s irreversibility milestone — PBFT’s $2f + 1$ **COMMIT**, the finalized Casper FFG checkpoint, Snowman’s counter- β acceptance — so every cross-protocol finality-latency claim is read from it. Throughout, time is the simulator’s model time; no number is a real-hardware claim, and published production figures are order-of-magnitude sanity checks (§1.4), not validation targets.

Tables 3.5 and 3.6 give the per-protocol latency/throughput and overhead/reliability formulas.

Table 3.5: **Latency and throughput per protocol.**

Metric	PBFT	Casper FFG	Snowman
<code>commit_latency_ms</code>	median per-node time to the first decided instance ($2f+1$ COMMIT)	median per-node time to the first finalized checkpoint (justify→finalize, ≥ 2 epochs)	median per-node time to counter- β acceptance of the first block
<code>tps</code>	decided ACUs per window (<code>decided_count</code> / <code>result.now</code>)	decided epochs per window (<code>decided_count</code> / <code>t_max</code>)	decided blocks per window (<code>decided_count</code> / <code>t_max</code>)
<code>goodput</code>	committed transactions per window (<code>committed_tx</code> / <code>time</code>)	committed transactions per window over finalized epochs	committed transactions per window

Throughput basis. As implemented, `tps` is a decided-event rate whose granularity is protocol-dependent — per block for PBFT and Snowman, per finalized epoch for Casper FFG — so it is not a like-for-like cross-protocol quantity. Cross-protocol throughput comparison therefore uses `goodput`, the committed-transaction rate, and never `tps`.

Table 3.6: **Overhead and reliability per protocol.**

Metric	PBFT	Casper FFG	Snowman
<code>consensus_msgs_per_acu</code>	<code>delivery_count</code> / <code>decided_count</code> , which evaluates to $(2n^2 - 2)/n = 2n - 2/n$; this is $O(n^2)$ per-instance traffic over an n -scaled decided-event denominator, not linear scaling	<code>delivery_count</code> / <code>decided_count</code> , measured $\approx 1.125n$ (un-aggregated all-to-all votes, $O(n^2)$ traffic; production BLS aggregation to $O(n)$ is not modeled)	<code>delivery_count</code> / <code>decided_count</code> ($O(K \cdot \beta)$ query/response deliveries per validator, independent of n)
<code>total_msgs_per_acu</code>	all deliveries per ACU; equals <code>consensus_msgs_per_acu</code> , as none of the three carries a separate mempool layer	as PBFT	as PBFT
<code>bytes_per_acu</code>	wire-byte budget per ACU; payload-dominated at the thesis workload — see note below	attestation + payload bytes per slot; payload-dominated	$O(K \cdot \beta)$ query/response bytes plus payload; payload-dominated
<code>success_rate</code>	0/1 indicator per run (1.0 iff an instance decided); becomes a frequency after <code>n_runs</code> aggregation	0/1 per run (iff an epoch finalized)	0/1 per run (iff a block reaches counter β)
<code>safety-violation rate (fork_rate)</code>	0 below threshold by construction; measured > 0 only above the 1/3 bound under equivocation	0 below threshold by construction; measured > 0 only above 1/3 under equivocation — a conflicting finalized checkpoint, not a reorg (LMD-GHOST is not modeled, §3.3.2 ④)	N/A — Snowman’s safety is probabilistic, reported via ε (empirical conflicting-decision rate against $(1 - \alpha_c/K)^\beta$); pre- β preference switches are convergence transients, not violations

Byte overhead. `bytes_per_acu` includes the 512-byte transaction payload on every transaction-carrying delivery; at the thesis workload this payload term dominates and amortizes, so `bytes_per_acu`/ n^2 *falls* with n rather than tracking each protocol’s message-complexity law. The RQ3 byte-overhead contrast is therefore read from `consensus_msgs_per_acu` (message count) or a payload-subtracted byte figure, not from raw `bytes_per_acu`.

The reliability family operationalizes the §2.1 properties. A *safety violation* is an observed breach of Agreement — two honest validators commit conflicting values at the same height in one run — measured by the safety-violation rate (recorded in the `fork_rate` column). For the deterministic-finality families it is 0 below threshold by construction and measured only above it. A *liveness failure* is an observed breach of Termination — at least one honest validator fails to commit within the measurement window — measured by the complement

of `success_rate`. Validity holds by construction and is not instrumented: the workload generator emits only well-formed transactions, and no module commits a value it did not receive.

Snowman is the exception to the zero-by-construction safety-violation rate: its finality is probabilistic rather than categorical, so the rate is reported instead as both the analytical bound $(1 - \alpha_c/K)^\beta$ (§3.3.3) and the empirical conflicting-decision rate across seeds. At the comparison baseline $\beta = 15$ that analytical bound is $\sim 10^{-15}$ at these n , so the empirical rate is unobservable in feasible seed counts. The empirical side is therefore collected only in a separate RQ4 safety regime at $\beta \in \{3, 5\}$, reported on its own and never placed on a cross-protocol throughput axis. Lowering β cuts Snowman’s $O(K \cdot \beta)$ cost and would otherwise manufacture a throughput advantage.

Protocols are compared under the common random numbers of §3.4.2. They share the network, arrival, and adversary-placement streams, so each cell measures the variance of the cross-protocol *difference*, not of either protocol alone. This is what makes a modest $n_{\text{runs}} = 20$ per cell sufficient, raised to 30 at the near-threshold Family C points where outcomes are most variable. Snowman’s internal poll sub-sampling has no cross-protocol counterpart, so its variance reduction is only partial.

Each trial writes one CSV row per `(protocol, scenario, seed)`, aggregated downstream to one row per configuration with the mean and a 95% confidence interval: Student-t for continuous metrics (small-sample, one degree of freedom below the seed count) and Wilson for rate metrics (`fork_rate`, `success_rate`), so a zero-violation cell is reported as $0/n_{\text{runs}}$ rather than a degenerate mean. Wilson stays honest at the boundary but does not narrow: even at 30 runs a zero-violation cell bounds the true rate only below ≈ 0.11 , so near-threshold safety verdicts are read as bounds, not point estimates.

Tables 3.5 and 3.6 list the columns populated at the honest baseline. Two column groups are defined in the schema but written only when the RQ4 adversarial sweep runs: the adversarial-threshold columns — `f_max_count` for PBFT and Snowman or the mutually exclusive `f_max_stake` for Casper FFG (the smallest adversary fraction at which a run’s safety or liveness invariant first breaks) — and the empirical and analytical Snowman safety columns discussed above. The finalized CSV layout is fixed in the project’s output-format specification.

3.6 Summary and threats to validity

The system model (§3.2) and metric schema (§3.5) absorb the three families’ structural asymmetries upstream of any experiment — through the ACU denominator, the per-protocol message accounting, and the per-protocol finality semantics — so no experiment sees them.

Three deliberate exclusions in the model bound the claims the chapter can support, and are stated here rather than left implicit.

- **No compute or bandwidth cost.** The latency-only network (§3.2) charges no per-byte, processing, or signature-verification cost. This favors the protocols whose dominant real-world cost is precisely what the model omits: PBFT’s $O(n^2)$ signature verification is charged for the messages and bytes it emits, which the simulator counts directly, but

not for the computation those messages imply. The bias is therefore confined to the latency and per-validator-cost verdicts; the message-count and byte-count comparisons are unaffected.

- **Synthetic open-loop workload.** The workload is Poisson arrivals of fixed-size transactions at a conflict rate of zero (§3.4.2), which isolates protocol behavior from workload-induced contention at the cost of not modeling real-traffic burstiness or double-spends, so conflict-driven reorganization lies outside the measured range.
- **Sub-production scale.** The validator-set sweep $n \in \{4, \dots, 25\}$ sits well below the production scale of the deployed protocols, Snowman’s in particular, so the RQ3 scaling verdicts are stated within this range and their extrapolation to production n rests on the sensitivity sweeps of §3.3.2 and §3.3.3 rather than on the sweep itself; the Snowman $n = 4$ point, where the rescaling rule degenerates to unanimity, is excluded outright.

Two further properties of the methodology are sources of caveat rather than exclusion.

- **Commensurability by convention.** The cross-protocol comparison is commensurable by convention, not by identity of the measured event (§3.5), so every comparative verdict is qualified by the conventions it rests on and is reported as robust only when it survives the governing sensitivity sweep.
- **Regime-coherence rules, not frozen knobs.** The protocols are held in their own regimes on the shared axes by two coherence rules — the Casper FFG slot-duration-to-delay pairing the runner enforces (§3.4.3) and the Snowman parameter rescaling that keeps one protocol across the sweep (§3.3.3) — rather than by freezing knobs that would place a protocol outside its design point.

The coverage of the comparison is itself bounded in three ways, each a consequence of what the chapter implements rather than of the model.

- **RQ4 fault survey scoped to three families.** The adversarial survey exercises only the three implemented families, so the RQ4 verdicts are scoped to those three.
- **Adversary grid covers twelve of eighteen catalogued pairs.** The sweep exercises the three generic capabilities — `delay-emission`, `withhold-participation`, and `equivocate-vote` — across the protocols, which is twelve of the eighteen valid (capability, protocol) pairs the catalogue defines. The remaining six are catalogued design space left unexercised, among them the entire leader-disruption surface — the documented pressure point of the leader-driven families, where a faulty primary or proposer forces leader rotation — so that surface is not measured here.
- **RQ5 synthesis traced over three families.** The cross-family Pareto synthesis of Chapter 5 is traced over the three families evaluated.

A `deadline stop`, finally, is scored as a liveness failure only against the in-window commit condition of §3.4.1, not by the clock alone. The model’s absence of a capacity ceiling (§3.4.2) and the threats named here are revisited in Chapter 6.

Chapter 4 reports the baseline, delay, and adversarial sweeps the matrix prescribes and answers RQ1–RQ4 against the schema fixed here.

Chapter 4

Results

4.1 Chapter roadmap

This chapter reports the experimental evaluation prescribed by the experiment matrix and answers research questions RQ1–RQ4 against the metric schema fixed in §3.5. It is organized by the three run families. §4.2 reports the baseline scaling sweep, in which validator-set size is the only independent variable and the network carries no injected delay or adversary. §4.3 reports the network-delay sweep, in which the network timeline varies while the validator set and workload hold fixed, and §4.4 the adversarial sweep, in which a fraction of the honest validators is replaced by each strategy of the fault model. Three protocols are evaluated throughout — PBFT, Casper FFG, and Snowman — consistent with the chapter scope set in §3.6.

The baseline serves two purposes. It establishes that each implemented protocol is correct on the honest path at every validator count, and it isolates the cost that scales with the validator set from the cost that the later sweeps will attribute to delay and to adversarial behavior. Because the baseline injects no delay and no faults, it is also the cleanest setting in which to confront the simulator’s measured numbers with each protocol’s published asymptotic theory.

4.2 Baseline: scaling with validator-set size

The baseline dataset sweeps validator-set size $n \in \{4, 7, 10, 16, 25\}$ at twenty seeds per configuration, driven by a common deterministic Poisson transaction workload (`offered_rate` = 100 tx/s, `tx_bytes` = 512, `conflict_rate` = 0), with common random numbers across protocols so that seed k presents the same arrival stream to every protocol. The result is 300 trials — fifteen scenarios of twenty seeds — aggregated to per-scenario means with 95% confidence intervals. Snowman is evaluated from $n = 7$ upward: at $n = 4$ its parameter rescaling collapses to the degenerate unanimity boundary $\alpha_c = K$ and is excluded from the comparison by the schema. Confidence intervals use a Student-t critical value at nineteen degrees of freedom rather than the Gaussian value named in §3.5; for a twenty-sample mean the distinction is small, and for the deterministic metrics discussed in §4.2.1 the interval is degenerate regardless of which critical value is used.

4.2.1 Statistical reliability

The dominant statistical fact of the baseline is that the seed has almost no effect. Across the twenty seeds of every scenario, commit latency, decision rate, message overhead, success rate, and fork rate each show a coefficient of variation of zero: their 95% confidence intervals are degenerate, of zero width. This is not an artifact of too few seeds but a property of the model. At the zero-delay honest baseline a protocol’s round structure and message counts are fixed once `(protocol, n)` is fixed; the only seeded randomness is the workload arrival process, and it perturbs only goodput, the rate of committed transaction bytes. Goodput accordingly carries the sole non-degenerate interval, with a coefficient of variation near 2.2% and a confidence half-width near 1% of its mean (Figure 4.1). Twenty seeds are therefore more than adequate. No larger seed set would narrow a deterministic metric’s interval, and the one stochastic metric is already estimated to within a percent. The confidence-interval exercise thus does double duty — it confirms determinism for the structural metrics and bounds workload noise for goodput.

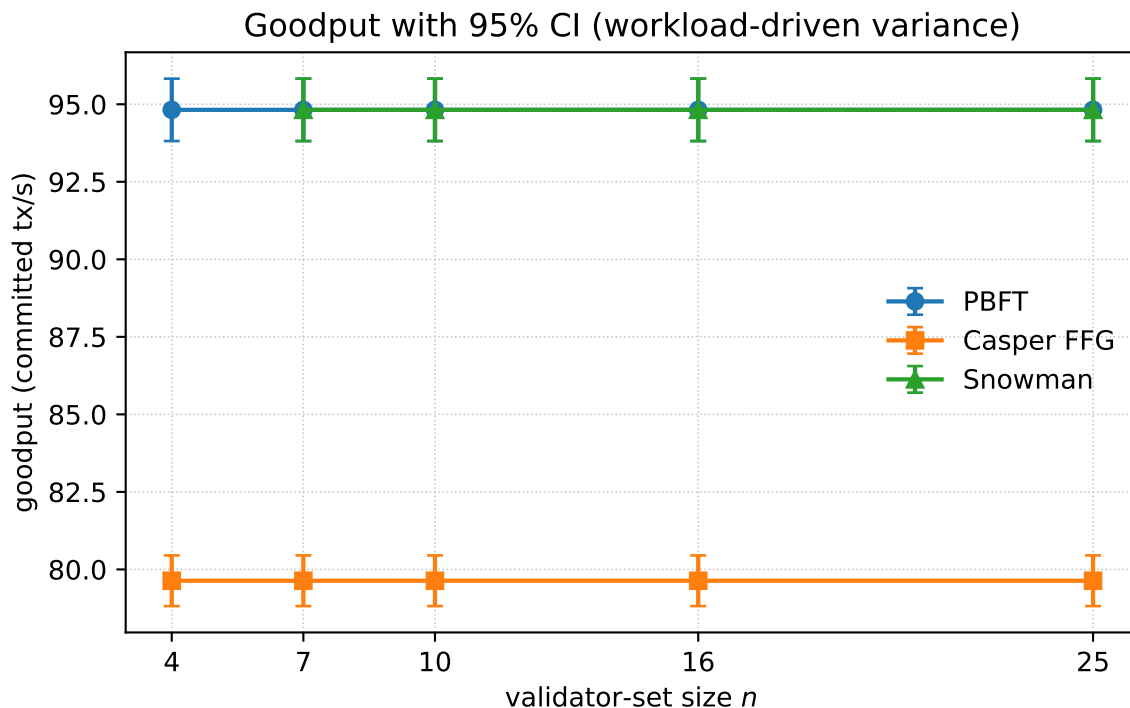


Figure 4.1: Goodput versus validator-set size with 95% confidence intervals — the one baseline metric carrying a non-degenerate interval.

4.2.2 Latency

Commit latency is flat in the validator count for all three protocols (Figure 4.2). PBFT and Snowman commit the first unit at approximately 1000 ms, and Casper FFG at approximately 5000 ms; none of the three changes measurably as n grows from 4 to 25. The explanation is that, absent network delay, latency is set by each protocol’s round and timer structure rather

than by the size of the validator set. PBFT’s figure reflects one proposal interval ahead of its three-phase commit; Snowman’s reflects one slot driving its repeated-poll counter; and Casper FFG’s reflects the justify-then-finalize rule, whose finality spans roughly two epochs at the configured one-second slot and two-slot epoch. The flatness is consistent with the theory that these protocols’ latency is round-bounded, but it must be read as a property of the zero-delay model: the latency cost that grows with the validator set, and that separates the protocols, surfaces only under the delay sweep of §4.3.

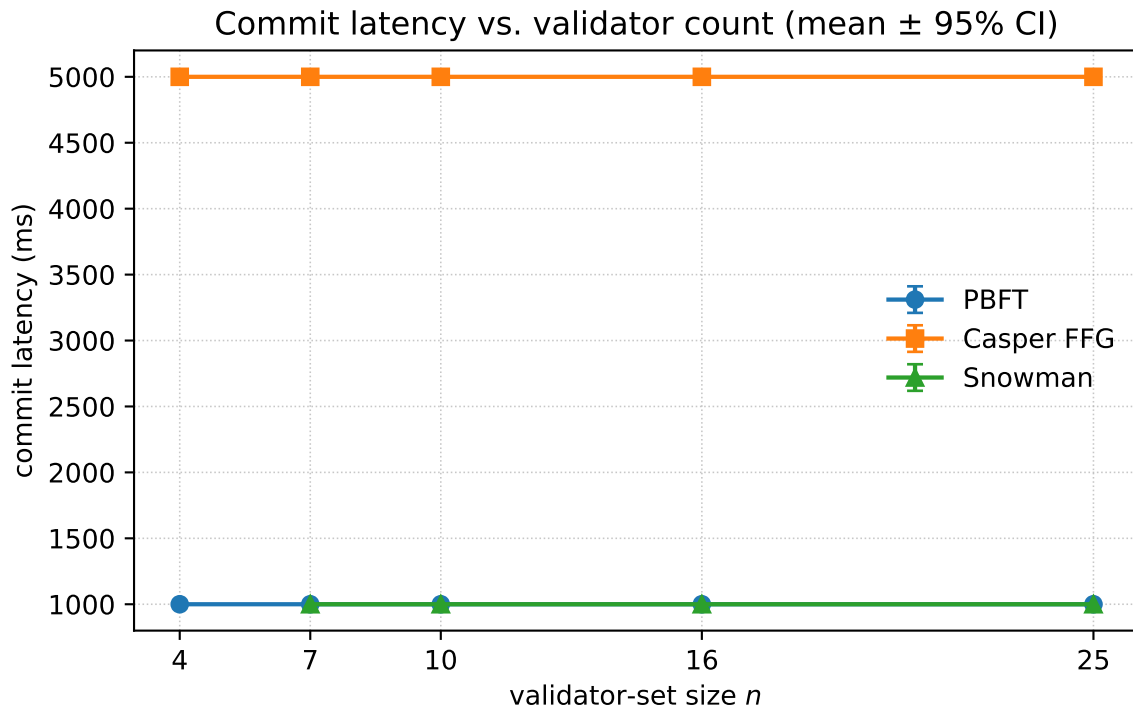


Figure 4.2: Commit latency versus validator-set size.

4.2.3 Throughput and goodput

The schema carries two throughput columns, and the baseline shows why the distinction matters. The decision rate `tps` grows linearly in n for every protocol — its per-validator value is constant at 0.95 for PBFT and Snowman and 0.40 for Casper FFG (Figure 4.3) — because `tps` counts decision events, of which each committed unit produces one per validator. It is therefore a decision-event rate that scales with the validator set by construction, not a measure of system transaction throughput. The honest throughput measure is goodput, the rate of committed transaction bytes, which is flat in n : approximately 95 tx/s for the per-block protocols PBFT and Snowman and approximately 80 tx/s for Casper FFG (Figure 4.4). The Casper FFG shortfall is a finality-tail effect: its per-epoch finality leaves the measurement window’s last unfinalized epoch uncommitted, a fixed end-of-window loss that the per-block protocols do not incur. The flat goodput is the expected result on a latency-only model with no per-transaction cost and no queue: offered load below the protocol’s cadence is always

absorbed. It must not be read as a measured capacity ceiling; saturation throughput requires a capacity model and is deferred.

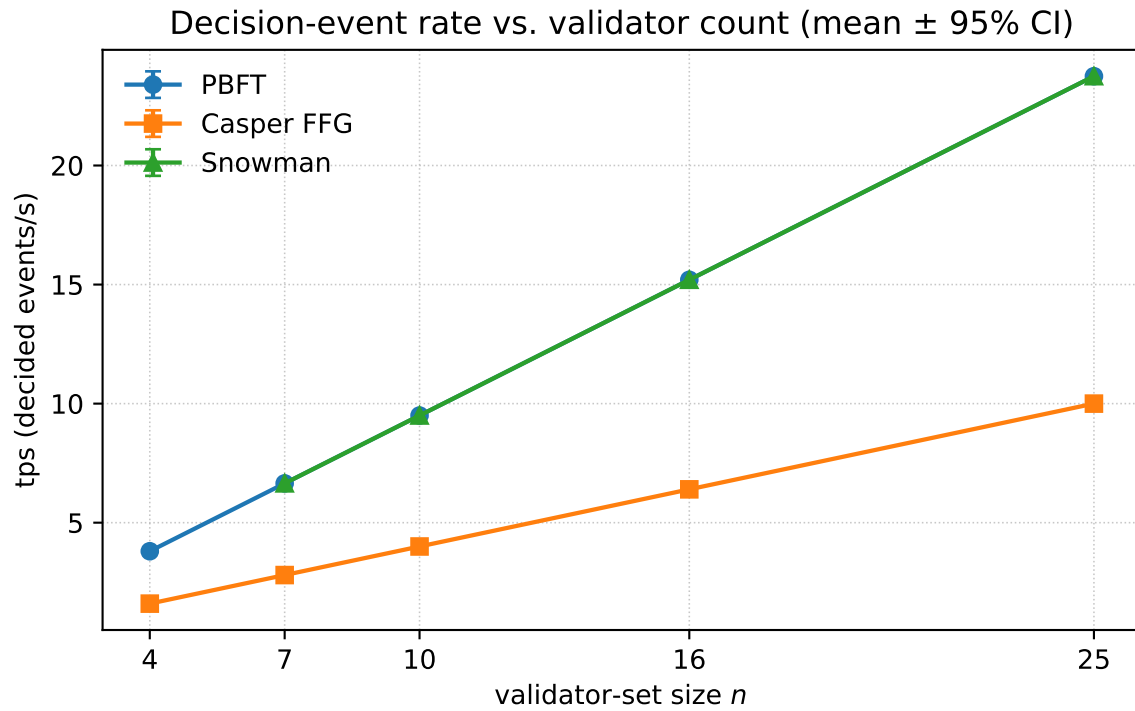


Figure 4.3: Decision rate (tps) versus validator-set size.

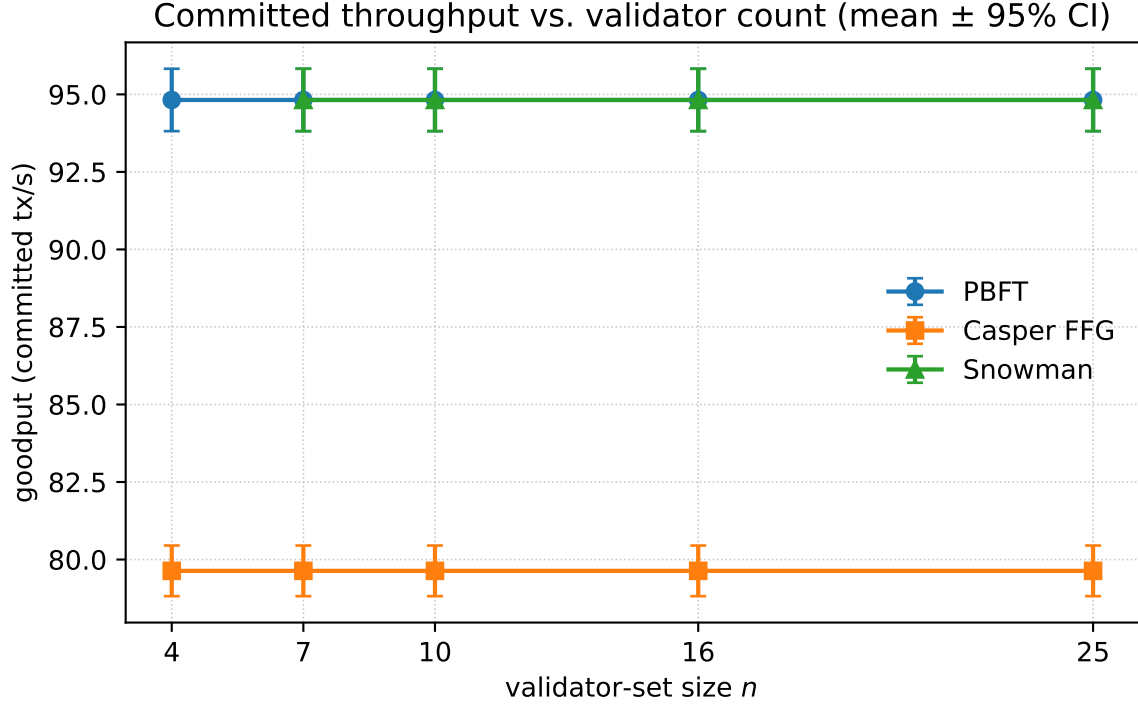


Figure 4.4: Goodput versus validator-set size, flat in n .

4.2.4 Communication overhead

Communication overhead is the metric on which the protocols separate most sharply, and it answers RQ3. Messages per committed unit grow with n for all three protocols, but the slopes differ by an order of magnitude (Figure 4.5, logarithmic axis). PBFT’s overhead approaches $2n$ messages per committed unit; Casper FFG’s approaches $1.2n$; and Snowman’s tracks $2 \cdot K \cdot \beta$, where K is the poll sample size and β the confidence threshold. Each measured trend matches the protocol’s published asymptotic cost: Figure 4.6 overlays the measured `total_msgs_per_acu` of each protocol on its predicted slope, and the markers fall on the prediction across the sweep, with the largest departures — near six to seven percent — confined to $n = 4$. PBFT’s normal-case cost is $O(n^2)$ messages per block; the per-unit metric reads $O(n)$ because the atomic-commit-unit denominator counts one decision per validator per instance, absorbing exactly one factor of n from the all-to-all prepare and commit phases. Casper FFG’s attestation phase is also all-to-all, and therefore $O(n^2)$ per epoch under the individually-signed-vote model that the original protocol specifies and that the simulator implements; its per-unit slope sits below PBFT’s not because of aggregation but because a single attestation phase serves more committed decisions per round than PBFT’s two broadcast phases. The production BLS aggregation that would reduce this cost to $O(n)$ is not part of the original Casper FFG specification and is not modelled here; introducing it, together with the corresponding threshold-signature variant of PBFT, is identified as future work in §6.3. Snowman’s measured overhead matches $2 \cdot K \cdot \beta$ to within half a percent across the sweep — the factor of two is the query-and-response pair of each poll — confirming the

per-validator $O(K \cdot \beta)$ cost that the Avalanche family is built around.

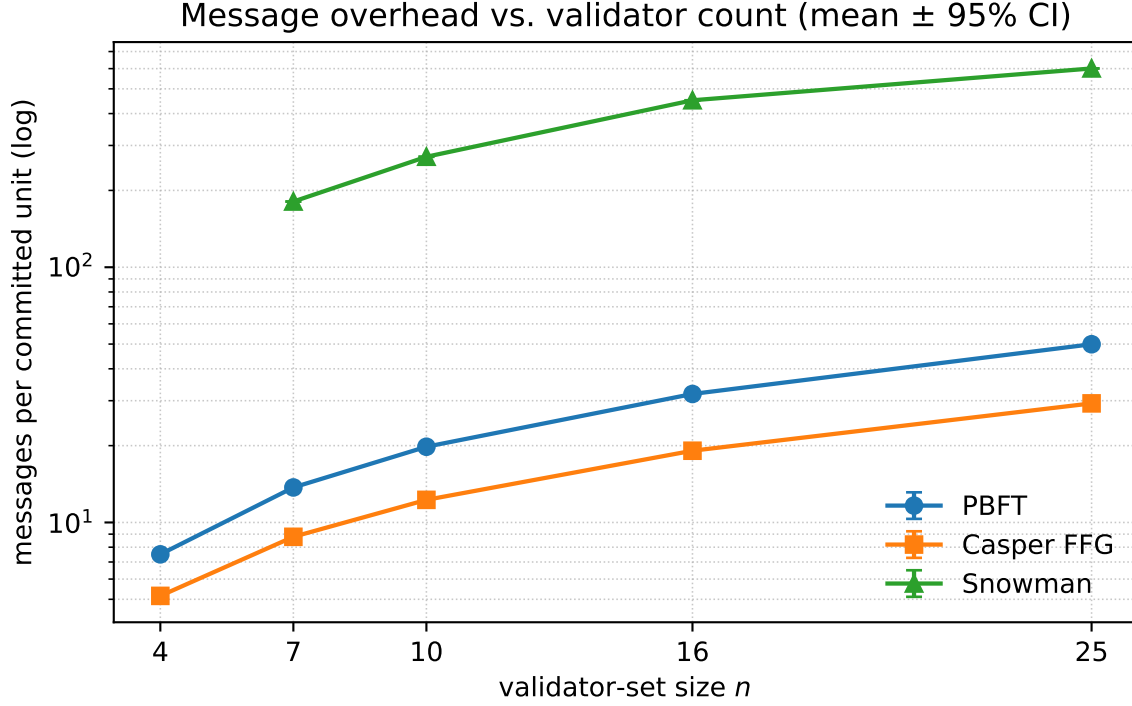
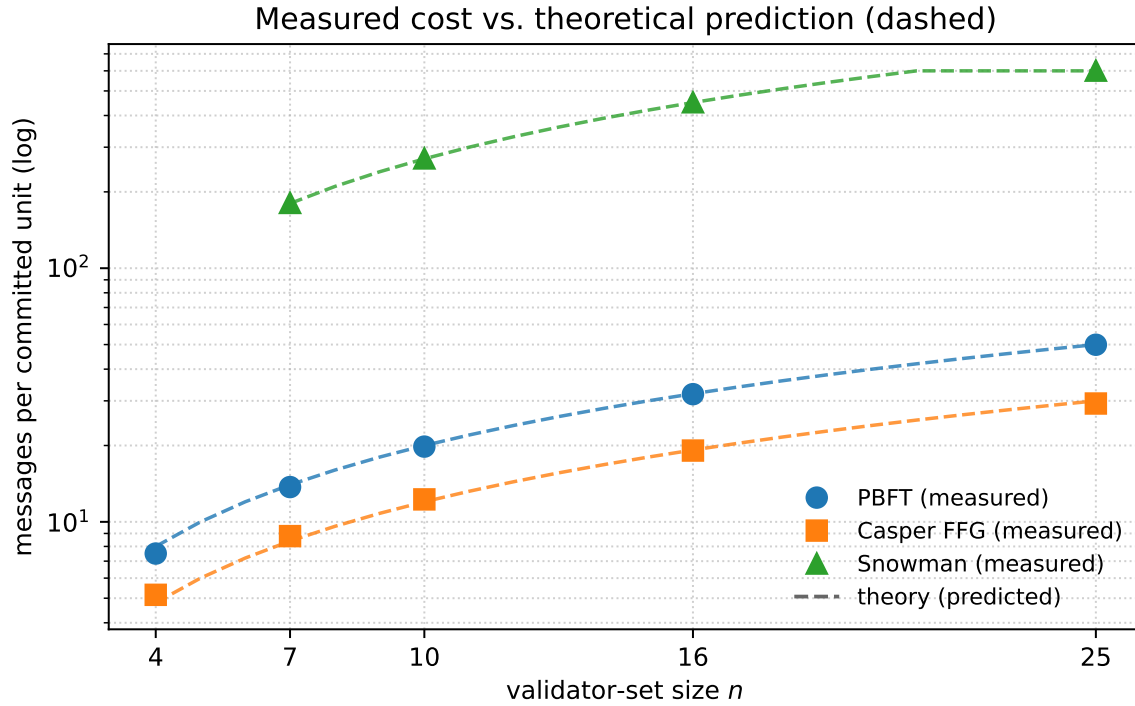


Figure 4.5: Message overhead per committed unit versus validator-set size, logarithmic axis.

Two readings of this result must be kept apart. Per committed unit, Snowman is the most expensive protocol by an order of magnitude — roughly twenty-six messages per validator against PBFT’s two — which is the price of repeated subsampling at thesis scale. Yet the property for which Avalanche is celebrated is that its per-validator cost is independent of n , and that is a statement about per-validator work, not about the network-aggregate `total_msgs_per_acu` plotted here. The aggregate necessarily grows with n because each of n validators performs the constant-per-validator work. The independence is further masked over this range because the thesis rescales $K = \min(20, n-1)$ to keep the protocol meaningful at small n , so K still tracks n until it saturates at the production value of 20 near $n = 21$. The comparison is therefore reported as a per-unit cost contrast, with the per-validator scalability stated separately so the figure is not misread.



markers = simulator; dashed = theory (PBFT $2n$, Casper $1.2n$, Snowman $2 \cdot K \cdot \beta$, $K = \min(20, n-1)$)

Figure 4.6: Measured message overhead against predicted asymptotic cost. Markers are the simulator’s measured `total_msgs_per_acu`; dashed lines are the per-protocol predictions — PBFT $2n$, Casper FFG $1.2n$, and Snowman $2 \cdot K \cdot \beta$ with $K = \min(20, n-1)$. Vertical axis logarithmic. The overlay is the visual form of the theory-match claim made in this section; the residual gaps at $n = 4$ are the finite- n corrections.

4.2.5 Reliability

Every scenario commits successfully and none forks: success rate is 1.0 and fork rate is 0.0 at every validator count for all three protocols (Figure 4.7). This confirms honest-path correctness — each protocol terminates and preserves agreement when no validator deviates — but carries no comparative information, since the three protocols are indistinguishable on both columns. The reliability metrics become discriminating only once the adversarial sweep drives validators past their fault thresholds, where the per-protocol safety invariants of §3.5 diverge; that analysis is §4.4.

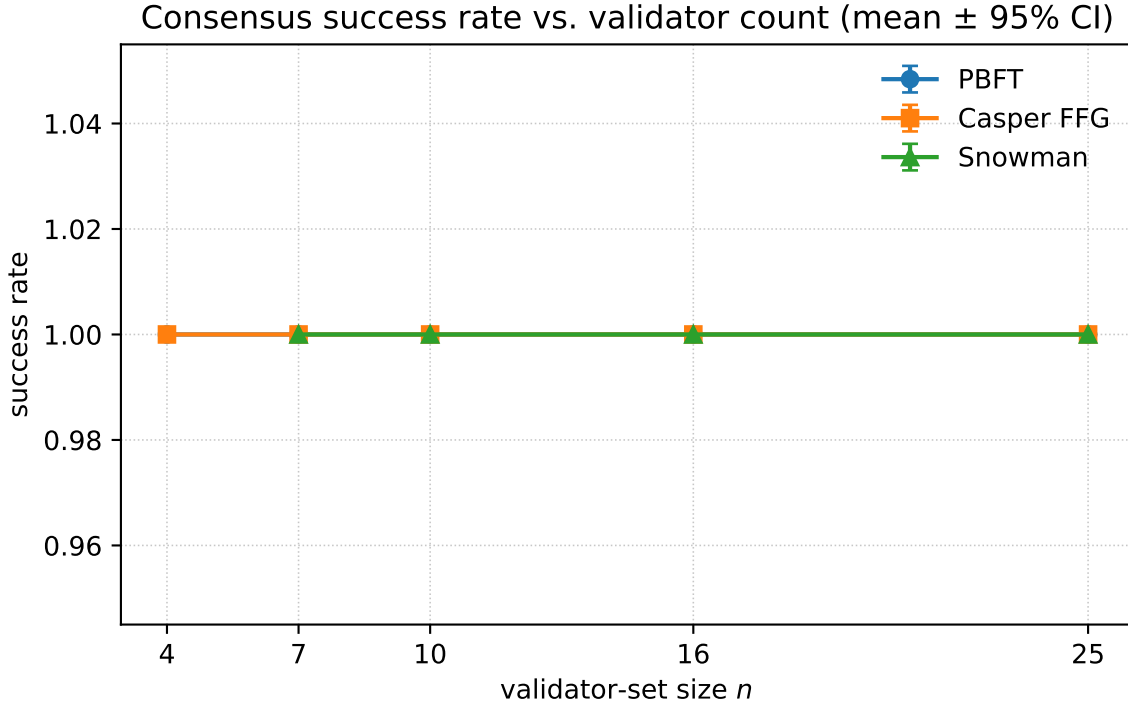


Figure 4.7: Reliability: success rate and fork rate versus validator-set size.

4.2.6 A note on the latency measurement point

The cross-protocol latency comparison of §4.2.2 — and of the delay sweep in §4.3 — is built from `commit_latency_ms`, and not from `finality_latency_ms`. Despite its name, `commit_latency_ms` is the canonical cross-protocol *time-to-finality* column: the median per-validator time to each protocol’s irreversibility milestone, its point of no return — the $2f+1$ commit quorum for PBFT, the finalized checkpoint after the justify-then-finalize rule for Casper FFG, and counter- β acceptance for Snowman. Aligning these three irreversibility milestones on one axis is what makes the comparison meaningful. The two columns coincide for Casper FFG and Snowman but diverge for PBFT, whose implementation adds a client-reply round so that its `finality_latency_ms` is measured one network hop past the internal commit quorum, at client-observed finality. Placing all three protocols’ `finality_latency_ms` on one axis would compare PBFT’s client-observed timestamp against the others’ internal timestamps, which is not a like-for-like comparison. The comparable-column choice is fixed in the schema page that governs figure construction rather than left to this prose, so the correctness of the comparison does not depend on the reader noticing this paragraph.

4.2.7 Baseline summary

Table 4.1: **Baseline means at $n = 25$ (production-scale end of the sweep), 20 seeds.** Latency and overhead are deterministic across seeds; goodput carries a 95% confidence interval.

Protocol	commit_latency_ms	goodput (tx/s)	total_msgs_per_acu	success	fork
PBFT	1000.0	94.82 ± 1.01	49.9	1.0	0.0
Casper FFG	5000.0	79.64 ± 0.82	29.3	1.0	0.0
Snowman	1000.0	94.82 ± 1.01	601.0	1.0	0.0

Three results carry forward to the later sweeps. All three protocols are correct on the honest path at every validator count. At zero delay both latency and goodput are flat in n , so neither metric separates the protocols here; the separation will come from the delay and adversarial axes. Communication overhead already separates the protocols by an order of magnitude in a direction that matches their published asymptotic costs, establishing the performance–structure contrast that RQ3 asks about and that the delay sweep of §4.3 will stress further.

4.3 Network-delay sweep

The delay sweep holds the validator set and the workload fixed and varies the network timeline, isolating the latency and message loss that the baseline of §4.2 deliberately excluded. It draws two regimes from run family B of the experiment matrix. The moderate regime applies two loss-free timelines of equal mean delay but different tail shape — **delay-uniform**, uniform on $[100, 500]$ ms, and **delay-exponential**, exponential with the same 300 ms mean. The heavy regime applies a heavy-tailed Pareto delay of roughly three-second mean, first without loss as a control and then under per-message drop probabilities of 5%, 10%, and 20%. Each cell is run at validator counts $n \in \{10, 25\}$ over twenty seeds with common random numbers across protocols, except the most expensive cell — Snowman at $n = 25$ under heavy delay — which is run over eight. As in §4.2, three protocols are covered.

Two properties of the measurement must be stated before the results. First, all cross-protocol latency figures are read from `commit_latency_ms`, the canonical time-to-finality column established in §4.2.6, so that each protocol’s irreversibility milestone is compared like for like. Second, delay and loss attack different properties and are reported apart: delay inflates time-to-finality and bears on RQ1, whereas loss erodes liveness and bears on RQ4.

4.3.1 Delay and time-to-finality

Under moderate delay the three protocols separate by more than an order of magnitude, and the separation is governed by each protocol’s round structure rather than by the network (Figure 4.8). PBFT rises from its zero-delay baseline of approximately 1000 ms to approximately 1.95 s, an increase of about 0.9 s that corresponds to one network round-trip absorbed into each of its three communication phases; the figure is near-flat in n because

those phases overlap across the validator set. Casper FFG rises from approximately 5.0 s to approximately 6.3 s, an increase of about 27%. This rise is slot-dominated rather than network-dominated: the experiment matrix couples the finality-gadget slot to the delay regime through the coherence rule $\text{slot} \geq 4 \cdot E[\text{delay}]$, rescaling the slot from one second to 1.2 s; the same 20% scales the slot-bound finality interval, lengthening FFG’s roughly five-second finality to about six seconds and leaving only some 0.3 s to attestation propagation. Casper FFG’s sensitivity to delay is therefore indirect, mediated by its slot clock.

Snowman is by far the most delay-exposed protocol, rising from approximately 1000 ms to between 12 and 15 s, a factor of twelve to fifteen. Its confidence counter requires $\beta = 15$ successful poll rounds in sequence, and each round is a query-and-response exchange that costs about two network delays, so the fifteen rounds accumulate roughly twelve seconds. Like the other two protocols its latency is near-flat — indeed slightly lower — in n , because the poll sample size K rescales with the committee rather than the first block’s finality time growing with it.

The two moderate timelines share a mean but differ in tail shape, and this distinguishes the protocols a second way. PBFT and Casper FFG are nearly insensitive to the tail, differing by at most three percent between the uniform and exponential timelines, because a fixed count of rounds or slots averages the per-message delay and washes out its distribution. Snowman is the exception: its exponential-timeline latency exceeds its uniform-timeline latency at both committee sizes — approximately 15.3 against 12.6 s at $n = 10$ — because each poll round waits on the slowest of its K sampled peers, and the memoryless tail inflates that slowest response and compounds the penalty across the fifteen sequential rounds. Communication overhead, by contrast, does not move with delay at all: PBFT’s messages per committed unit land on the same $2n$ value measured at zero delay and Snowman’s within about two percent, confirming that message count is fixed by protocol structure and not by network timing, consistent with the overhead analysis of §4.2.4.

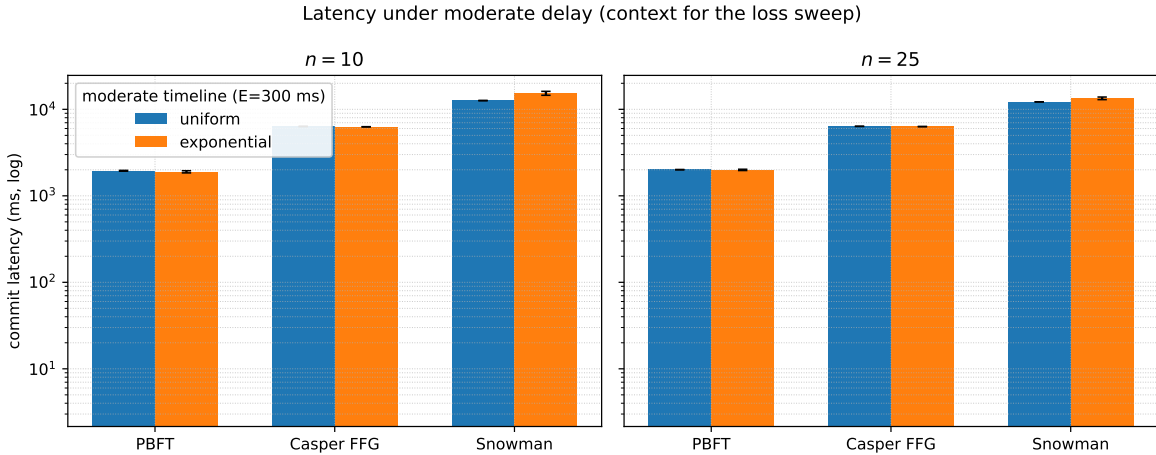


Figure 4.8: Commit latency under moderate delay. Median per-validator `commit_latency_ms` under the two equal-mean moderate timelines (`delay-uniform` and `delay-exponential`), grouped by protocol and faceted by validator count; logarithmic vertical axis.

4.3.2 Packet loss and the resilience ranking

Under packet loss the question shifts from how long finalization takes to whether it happens at all. The measure is the finalization rate: the number of instances a protocol finalizes under loss as a fraction of the number it finalizes on the matched loss-free control. Figure 4.9 plots this rate against the drop probability for each protocol and committee size, and three distinct degradation shapes are visible: PBFT declines along a shallow tail that stays above zero to the deepest tested loss, Snowman holds a high plateau and then falls to near-zero within a single loss step, and Casper FFG collapses at the first loss step. A fixed 95%-finalization breakpoint was considered as a resilience score and rejected, because every protocol is already below that threshold at the lightest tested loss, so the threshold collapses the field to an uninformative tie.

The protocols are therefore ranked by the area under the finalization-rate curve (AURC), with survival depth — the deepest loss at which a protocol still finalizes anything — as the tiebreak, and the two committee sizes reported separately. Figure 4.10 shows the ranking directly. At $n = 10$ the order is strict: PBFT leads with an AURC of 0.253 and survives to 20% loss, Snowman follows at 0.174, and Casper FFG trails at 0.149. At $n = 25$ PBFT and Snowman are a statistical tie at the top: Snowman’s higher point estimate of 0.369 and PBFT’s of 0.351 carry confidence intervals that overlap — $[0.366, 0.372]$ against $[0.327, 0.376]$ — so neither can be ranked above the other, while Casper FFG remains last at 0.140. Unlike the baseline of §4.2.1, where every structural metric was deterministic across seeds and its interval degenerate, the finalization rate genuinely varies with the seed, because which messages are dropped depends on it; the intervals in Figures 4.9 and 4.10 are accordingly non-degenerate and carry information. The interval for Snowman at $n = 25$ is the widest, as that cell is estimated from eight seeds rather than twenty.

Two findings stand out from the ranking. The first is that PBFT is the only protocol still finalizing at 20% loss, at both committee sizes — 10% of its control rate at $n = 10$ and 6% at $n = 25$ — whereas neither other protocol survives past 10% loss, each having fallen to zero by the 20% step. The second is that committee size is a sharp resilience lever for Snowman: its finalization rate at 5% loss rises from 0.195 at $n = 10$ to 0.904 at $n = 25$, so that at light loss the larger committee makes Snowman the strongest of the three, even as it still cliffs to near-zero by 10%. The same enlargement is a mild liability for Casper FFG, whose rate at 5% loss falls slightly, from 0.070 to 0.051. Throughout the loss sweep no protocol ever forks: the fork rate is zero on every row, so what loss degrades is liveness — the ability to finalize — and not safety.

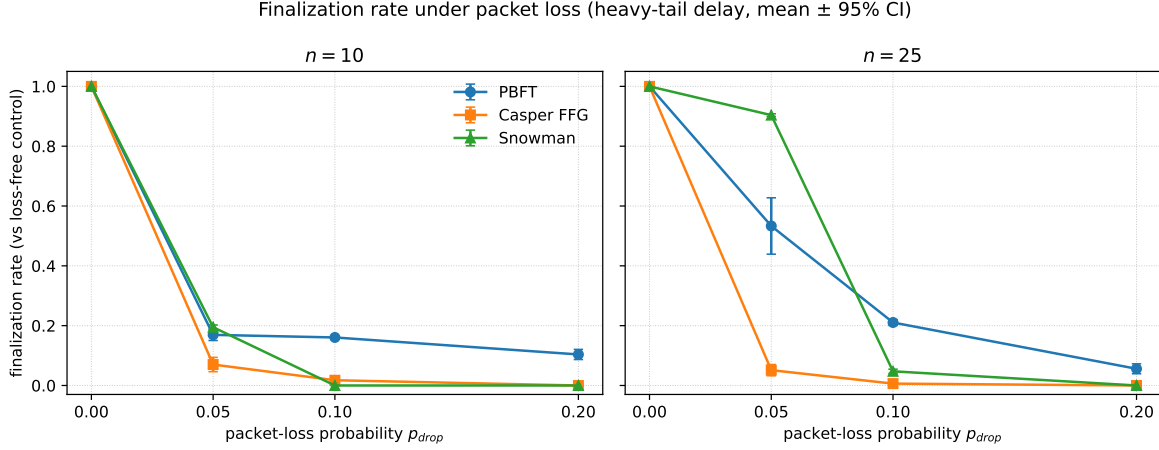


Figure 4.9: Finalization rate under packet loss. Finalization rate against per-message drop probability, one curve per protocol, faceted by validator count, with 95% confidence intervals.

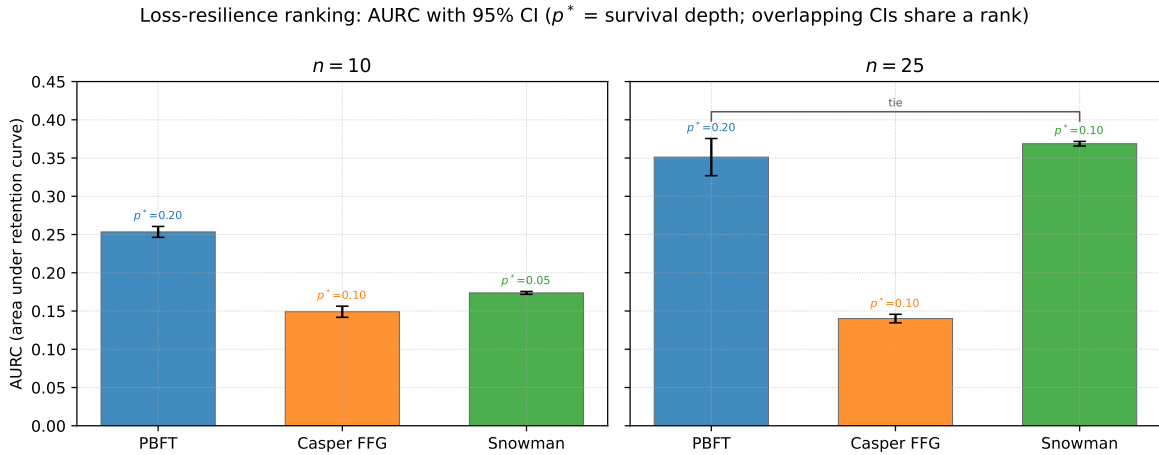


Figure 4.10: Loss-resilience ranking. Area under the finalization-rate curve (AURC) with 95% confidence intervals, annotated with each cell’s survival depth p^* , faceted by validator count; protocols whose intervals overlap share a rank.

4.3.3 Mechanisms of degradation

The ranking is explained by what each protocol can do when messages are lost (Figures 4.11 and 4.12). PBFT is the most robust because it has a genuine recovery path. When dropped prepare or commit messages stall an instance below its quorum, a per-instance timer fires, the replicas rotate the leader through a view-change, and the instance is reissued under the new leader. This is a retry rather than a retransmission — a fresh round with fresh messages, and so a fresh opportunity to assemble the quorum — and enough retries eventually succeed even under heavy loss. The recovery work is visible in the view-change count, which climbs with the loss level from zero to sixteen and then thirty at $n = 10$, and from zero through twenty-eight and sixty-three to seventy-five at $n = 25$ (Figure 4.12).

Snowman’s robustness is of a different kind: redundancy within a single poll round rather than recovery across rounds. A round closes once α_c agreeing responses arrive, so it tolerates the loss of peers beyond that threshold, but there is no poll timeout and no retransmission, and a round that never collects α_c responses simply stalls. Because a response survives only if both its query and its reply survive, the expected number of usable responses per round is $K \cdot (1 - p)^2$, and the round closes only while this exceeds α_c . The slack $K - \alpha_c$ is one at $n = 10$ but four at $n = 25$, which is why the larger committee tolerates so much more loss. Once the per-round margin is exhausted, however, the $\beta = 15$ rounds compound: the probability of completing all fifteen falls away sharply, turning the degradation into a cliff rather than a slope. This compounding argument predicts the location of the cliff and the committee-size ordering but not the exact rate, and is used qualitatively.

Casper FFG is the most fragile because it has neither recovery nor in-round redundancy. Finalization requires a supermajority of at least two-thirds of stake to attest the same checkpoint, and then two such justifications in consecutive epochs; attestations are broadcast once per epoch, so when enough are lost the epoch never justifies, and there is no leader to rotate and no resampling to fall back on. A 5% drop already collapses it, and a larger committee is a slight liability because it adds attestation links that can be lost without adding redundancy. Under heavy delay a node may justify an epoch before its checkpoint block arrives locally; a guard added during the heavy-delay experiments converts what would have been a crash into an honest stall, in which the node skips the attestation and retries at a later slot.

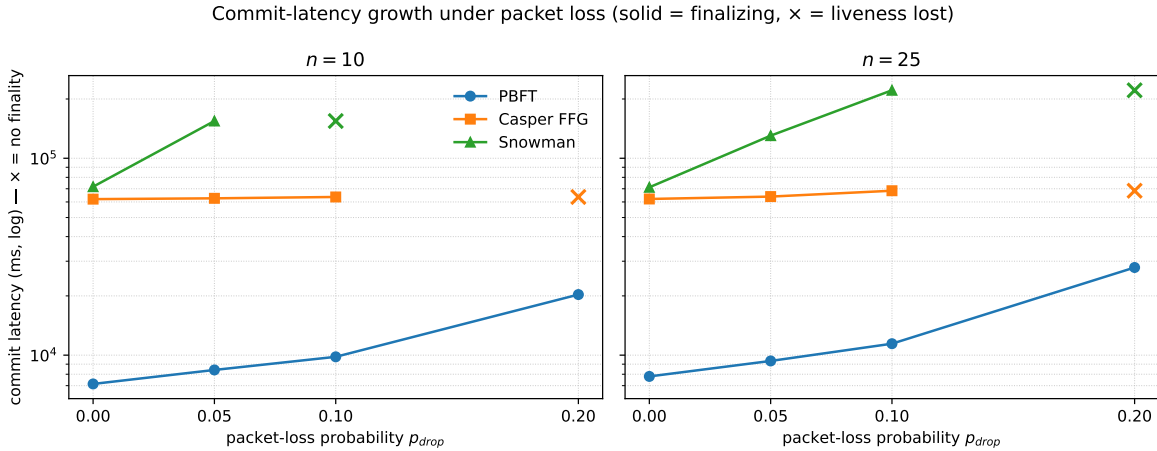


Figure 4.11: Commit-latency growth under loss. `commit_latency_ms` against drop probability on a logarithmic axis; a solid line marks the levels at which a protocol still finalizes and a cross marks the level at which liveness is lost.

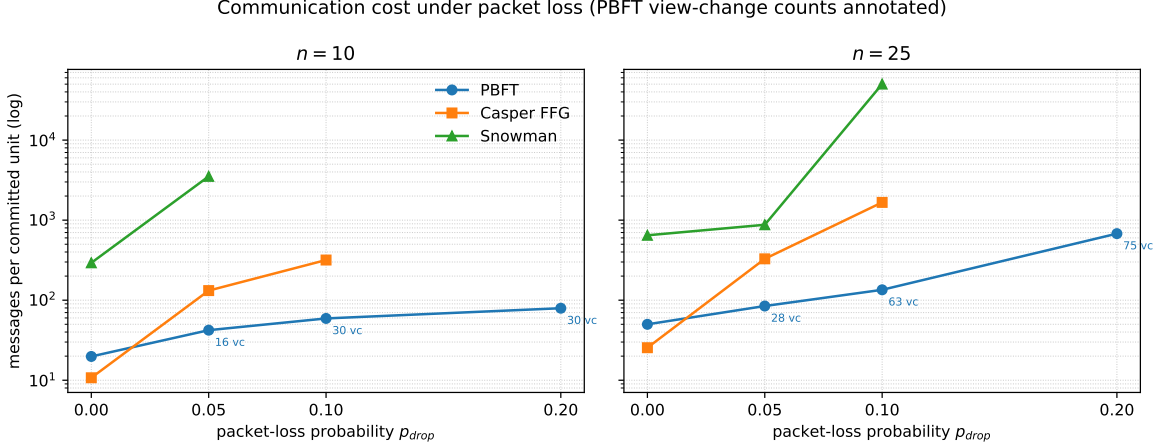


Figure 4.12: Communication cost of survival. Messages per committed unit against drop probability on a logarithmic axis, with PBFT’s view-change counts annotated.

4.3.4 The latency–liveness tradeoff

The two stress axes converge on one result: the protocols that survive loss are the ones that pay the most latency to do so (Figure 4.13). PBFT and Snowman both inflate their time-to-finality by factors of roughly two to three-and-a-half at the worst loss they survive, and they convert that cost into survival — PBFT into a long tail to 20% loss, Snowman into a high but brittle plateau. Casper FFG does not make this trade: it inflates latency by only about three to ten percent, but that near-constant cost is measured over the few seeds that still finalize at all, and it buys almost nothing, since the protocol no longer finalizes by 10% loss. No configuration in the dataset is both cheap and resilient; the operator-facing reading is that protecting liveness against a lossy network is a choice of how much latency to spend, not whether to spend it.

The verdict for the delay family follows. PBFT degrades most gracefully, declining smoothly and remaining alive at the deepest tested loss at both committee sizes; Snowman is strong but brittle, best in class at light loss when the committee is large but prone to sudden collapse; and Casper FFG is fragile, never establishing a resilient plateau. The $n = 25$ tie between PBFT and Snowman is a genuine crossover rather than measurement noise: the two protocols win on different virtues — Snowman on the area under the curve, having retained 0.90 finality at 5% loss, and PBFT on survival depth, being alone alive at 20% — so neither ranks first outright. The cross-regime question of whether any one family dominates across baseline, delay, and adversarial conditions is deferred to the synthesis of Chapter 5 (RQ5).

Two caveats qualify these results. The first concerns the nature of the finality being timed. The aligned milestones of §4.2.6 differ in kind. PBFT and Casper FFG offer deterministic finality. Casper FFG’s is additionally accountable, in that reverting a finalized checkpoint requires at least one-third of the stake to be slashed. Snowman’s finality is probabilistic, with a residual reversion probability bounded by $(1 - \alpha_c/K)^\beta$ — approximately 5×10^{-15} at $n = 10$ and a looser 3×10^{-11} at $n = 25$, the bound loosening at the larger committee because the rounding in $\alpha_c = \lceil 0.8K \rceil$ raises the ratio above 0.8 only at small K . The empirical and

analytical ε columns recording this residual per run are deferred to a separate sweep at a deliberately weakened confidence depth, as §4.4.3 details. The second caveat is the more important for interpreting the loss results: loss is modeled as permanent per-message drop with no transport-layer retransmission, so the finalization-rate curves are an upper bound on fragility. The ordering — PBFT most robust, Casper FFG most fragile — is a property of the protocols’ recovery mechanisms and would survive a retransmitting transport, but the absolute collapse levels of 5% to 20% would shift higher beneath one.

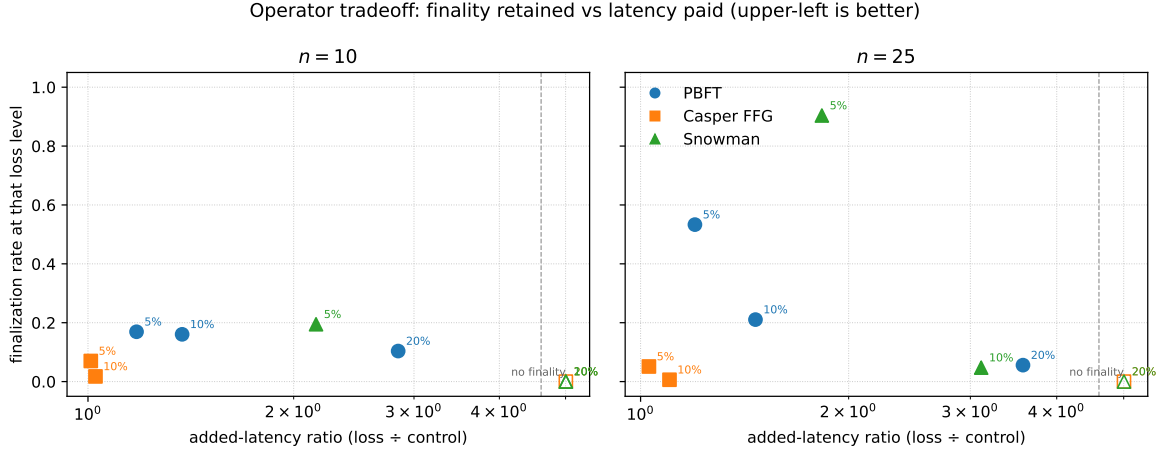


Figure 4.13: The operator tradeoff. Finalization rate retained against the added-latency ratio (loss latency divided by control latency, logarithmic), faceted by validator count; cells that no longer finalize are pinned on a “no finality” band, and the operator-best region is the upper left.

4.4 Adversarial sweep

The adversarial sweep holds the network at a constant baseline delay and replaces a fraction of the honest validators with a single Byzantine strategy, isolating adversarial behavior from the network effects of §4.3. It draws run family C from the experiment matrix and exercises the three generic capabilities of the adversary catalog in turn — delayed voting, silent non-participation, and equivocation (defined in §3.4.2). Each is swept from an honest control through a band of injected adversarial fractions φ , at validator counts $n \in \{10, 25\}$, twenty seeds per cell under common random numbers. Here φ is the swept variable, distinct from the tolerated threshold f , and is denominated in each protocol’s natural unit: replicas for PBFT and Snowman and stake for Casper FFG (§3.4.2). The sweep is extended past the one-third threshold where a safety failure is possible, namely equivocation against PBFT and Casper FFG. The paired committee sizes separate size-invariant results from size-dependent ones such as Snowman’s silence cliff (§3.3.3). As in §4.2 and §4.3, the survey covers the three implemented protocols.

The sweep reports the two outcome families RQ4 separates: liveness, measured as the consensus success rate — the fraction of seed-runs that finalize within the measurement window (§3.5) — and safety, reported as a per-protocol safety-violation rate, except for

Snowman, whose probabilistic finality is reported through its analytical bound ε rather than a fork count.

4.4.1 Delayed voting

Under delayed voting the three protocols separate by failure mode, and the split follows protocol structure rather than the size of the delayed set (Figure 4.14). PBFT is immune: its delayed validators are backups, the honest remainder already meets the $2f + 1$ prepare and commit quorums, and the view-0 primary commits without rotation, so time-to-finality holds at its baseline — a ratio of 1.0 against the honest control — and the success rate stays at 1.0 to the deepest fraction tested, $\varphi = 0.30$, with zero view-changes. This is the honest-leader case: the delayed set spares the view-0 primary, so a leader-targeting adversary is a separate surface, returned to in §4.4.4. Casper FFG keeps its finality latency unchanged when it finalizes, because finality is gated on a stake supermajority the honest validators still form, but its liveness degrades: the per-slot proposer rotates, a delayed validator is periodically the proposer, and the block it owes stalls for that slot, dropping the success rate to a worst pooled 0.60 at $n = 10$ and 0.65 at $n = 25$. Snowman is the costliest case: it neither forks nor stalls, so the success rate holds, but its time-to-finality explodes by a factor of roughly 62 at $n = 10$ and 49 at $n = 25$ against the honest control, because each of its β sequential poll rounds waits on the slowest sampled peer and a delayed peer inflates every round it is sampled into. The blow-up becomes severe near $\varphi = 0.20$.

These three outcomes illustrate the performance–security tradeoff. The two protocols that finalize without liveness loss do so for opposite reasons and at opposite costs: PBFT’s fixed quorum tolerates slow backups for free, while Snowman’s sampled supermajority tolerates them only by waiting, paying the latency penalty quantified above for the same survival. Casper FFG pays no latency penalty but loses liveness instead, its single rotating proposer having no redundancy when the proposer is the slow node. Ranked by the liveness held against the adversary, PBFT and Snowman tie ahead of Casper FFG, with PBFT first on the finality-cost tiebreak. None of the three forks: delayed voting threatens liveness and latency, never agreement.

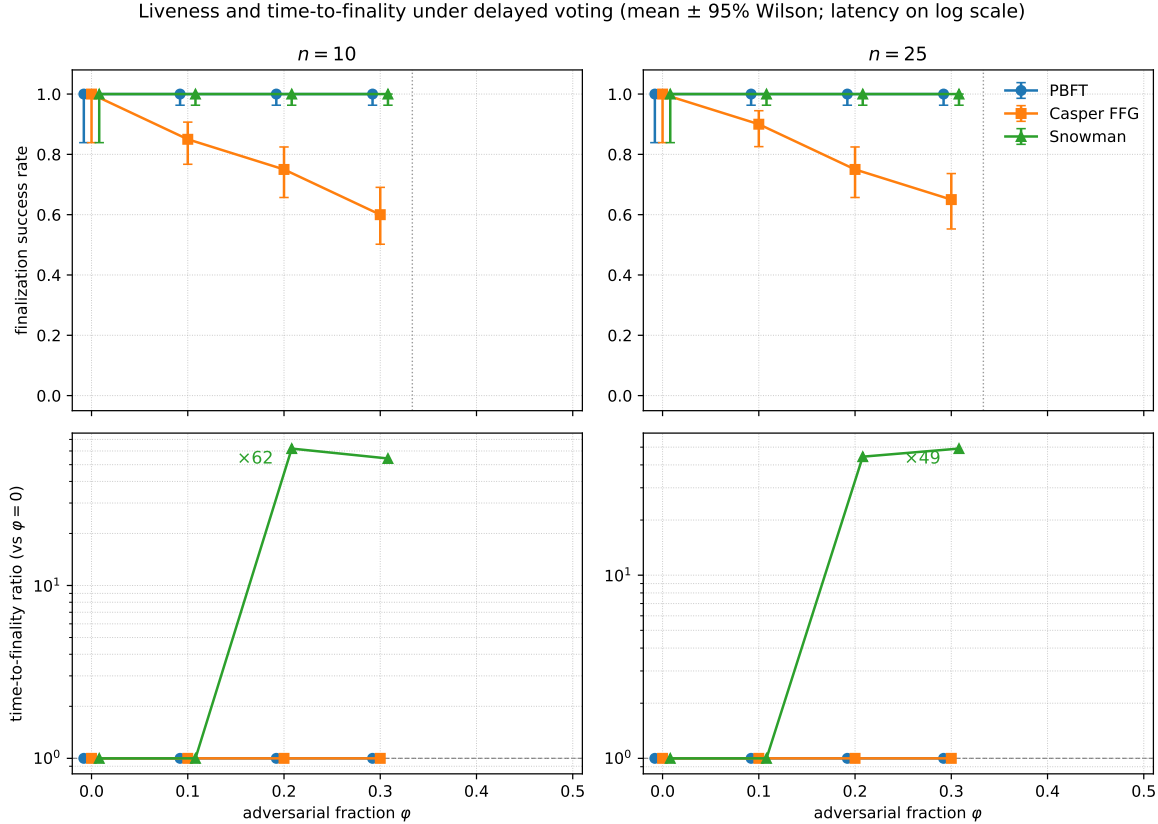


Figure 4.14: Liveness and time-to-finality under delayed voting. Top row: finalization success rate against the injected adversarial fraction ϕ , faceted by validator count, on which PBFT and Snowman coincide at 1.0 while Casper FFG dips. Bottom row, on a logarithmic scale: the time-to-finality ratio against the honest control, which separates the two protocols the top row leaves indistinguishable — PBFT and Casper FFG hold at $1.0\times$ while Snowman pays a blow-up of roughly $\times 62$ at $n = 10$ and $\times 49$ at $n = 25$.

4.4.2 Silent non-participation

When the adversarial validators go silent rather than slow, the question becomes how deep a fraction each protocol can lose and still finalize (Figure 4.15). The protocols are ranked by survival depth — the deepest ϕ at which a protocol still finalizes anything — rather than by the onset of degradation, because the two keys order Casper FFG and Snowman oppositely, and survival is the faithful measure of where liveness fails. PBFT exhibits a clean quorum cliff: it finalizes with no throughput loss up to $\phi = 0.33$ and dies at $\phi = 0.40$, the point at which the silent set drops the honest remainder below the $2f + 1$ quorum. Casper FFG degrades gracefully over the same range, still finalizing at $\phi = 0.33$; its throughput — the rate of committed units, a separate magnitude from the success rate Figure 4.15 plots — decays in proportion to the participating stake, approximately $1 - \phi$ (Figure 4.20), to a worst surviving ratio near 0.49 at $n = 10$ and 0.47 at $n = 25$. Snowman cliffs earliest, and its cliff is committee-size-dependent: it survives only to $\phi = 0.10$ at $n = 10$ and $\phi = 0.20$ at $n = 25$,

because a poll round closes only once α_c sampled peers respond and never completes when too many are silent. At $n = 25$ the $\varphi = 0.20$ cell is technically alive but starved, finalizing at roughly half a percent of its control throughput.

The ordering is therefore PBFT and Casper FFG ahead of Snowman, the two leaders tied on survival depth and separated only by PBFT’s undegraded throughput below its cliff. As under delayed voting, no protocol forks: silence erodes liveness, not safety. The result inverts the delayed-voting verdict for Snowman: the protocol that best tolerated slow validators, albeit at a latency cost, is the least tolerant of silent ones, because a sampled supermajority can wait out a slow peer but cannot complete a poll around an absent one.

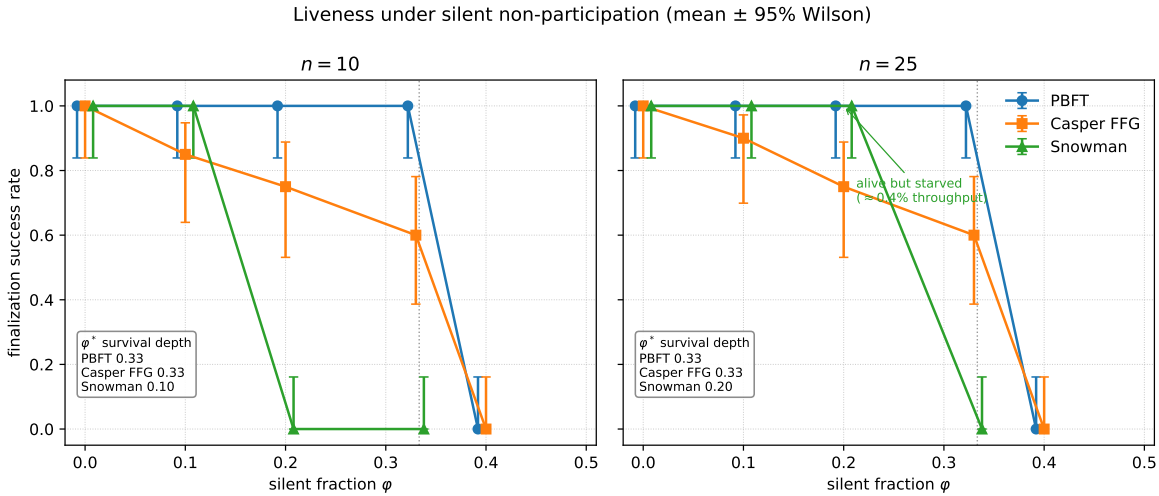


Figure 4.15: Liveness under silent non-participation. Finalization success rate against the injected silent fraction φ , drawn as step cliffs, one curve per protocol and faceted by validator count, with each protocol’s survival depth φ^* — the deepest fraction at which it still finalizes — boxed. The cliffs place PBFT and Casper FFG ahead of Snowman; the Snowman cell that remains alive at $n = 25$, $\varphi = 0.20$ is labelled to mark that it finalizes only at a collapsed fraction of its control throughput.

4.4.3 Equivocation

Equivocation is the only one of the three strategies that can break safety, and it is the axis on which PBFT and Casper FFG are driven past the one-third threshold to expose the breaking point. Below the threshold all three protocols preserve agreement and, with one exception, liveness (Figure 4.16): Casper FFG’s success rate holds across the full swept range to $\varphi = 0.50$, Snowman’s to the top of its grid at $\varphi = 0.33$, while PBFT’s dips through a view-change window before an apparent recovery above the threshold — a recovery that is the safety failure itself, not restored liveness. Each protocol holds its safety invariant to $\varphi = 0.33$; they differ entirely in what happens above it, and the difference is the kind of failure, not its onset.

PBFT fails catastrophically and without accountability. Below the threshold its view-change mechanism absorbs the equivocation — an equivocating primary is detected and the

replicas rotate to a new leader — and the view-changes multiply with the equivocator fraction (Figure 4.17), reaching 10 rotations at $n = 10$ and 25 at $n = 25$ at the last safe fraction. At $\varphi = 0.40$ the equivocating set exceeds what rotation can contain, and two honest replicas commit conflicting values at the same height: the safety-violation rate steps from zero to a deterministic breach, with 229 conflicting instances at both committee sizes (Figure 4.18). The fork is deterministic. It is invariant across seeds because the equivocating set is fixed rather than sampled. It is identical in count at both committee sizes because the number of conflicting (`view`, `seq`) instances inside the measurement window is set by the proposer’s round cadence and the window length, not by the validator-set size. And it is unaccountable: PBFT carries no mechanism to attribute the conflicting commit to the validators that caused it.

Casper FFG never forks in the model, and when it fails it fails accountably. Its finality rule admits no two conflicting finalized checkpoints below the threshold, and above it the failure surfaces not as a fork but as slashable stake: the stake an adversary must expose to force a conflicting checkpoint rises with the equivocator fraction and crosses the one-third accountability line at $\varphi = 0.40$, peaking at 0.50 of stake at $n = 10$ and 0.48 at $n = 25$ (Figure 4.19). This is the accountable-safety property of the finality gadget: a safety violation is possible only at the cost of at least one-third of the stake being provably slashable [18]. Casper FFG is also the most liveness-robust of the three to equivocation, its success rate holding across the full swept range to $\varphi = 0.50$.

Snowman presents no fork surface at all. Equivocation against a subsampling protocol reduces to a lying responder, with no more effect than withholding a response (§3.4.2), so the strategy is not swept above the threshold for Snowman and the empirical safety-violation rate is zero on every cell of the grid. Snowman’s safety is probabilistic rather than categorical and is reported through its analytical bound $\varepsilon \leq (1 - \alpha_c/K)^\beta$, approximately 5×10^{-15} at $n = 10$ and a looser 3×10^{-11} at $n = 25$ — the bound loosening at the larger committee because the rounding in $\alpha_c = \lceil 0.8K \rceil$ lifts the ratio α_c/K above 0.8 only at small K (§3.3.3). An empirical zero cannot confirm a bound this small: with eighty observations per cell the data bounds the violation rate only below a few percent, many orders of magnitude above the analytical ε , so the measured zero records that no violation occurred at the baseline confidence depth $\beta = 15$, not that the bound has been observed. Witnessing ε directly would require a separate sweep at a deliberately weakened confidence depth.

Ranked by safety, the order is Snowman, Casper FFG, PBFT. All three hold to $\varphi = 0.33$; the ordering is not about which fraction each tolerates but about what failure occurs above it: Snowman has no fork to suffer, Casper FFG forks only at a provable cost of one-third of the stake, and PBFT forks deterministically and silently. Two cautions attach. The three failures sit on incommensurable scales — a conflicting-commit rate for PBFT, a slashable-stake fraction for Casper FFG, and an analytical bound for Snowman — so the ranking compares kinds of failure, not magnitudes on a single axis. And Snowman’s first place is in part structural: a subsampling protocol presents no fork-inducing surface to begin with, which is why the strategy is swept only to $\varphi = 0.33$ for Snowman; it is ranked first for having no safety failure mode to expose, not for surviving a fraction at which the others break (§3.4.2).

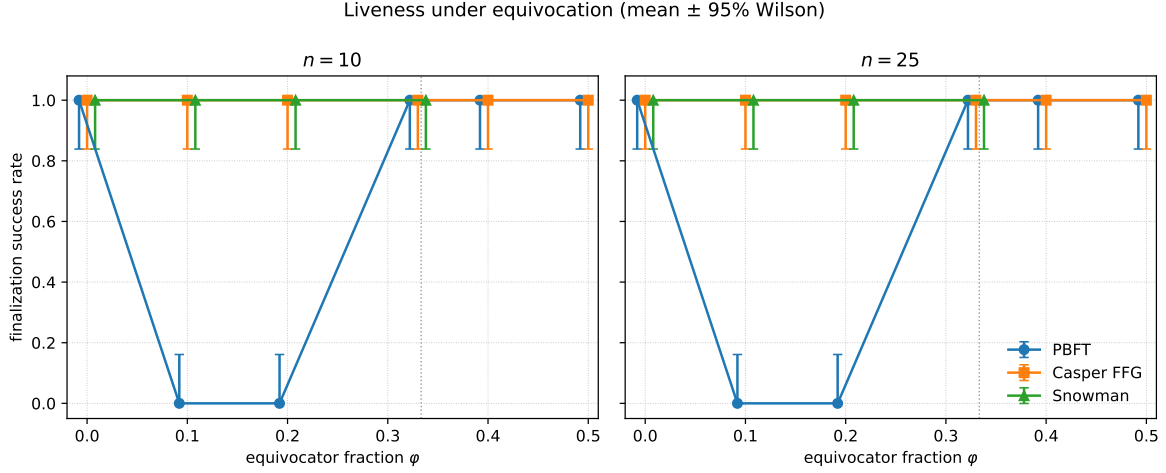


Figure 4.16: Liveness under equivocation. Finalization success rate against the equivocator fraction φ , one curve per protocol, faceted by validator count; PBFT’s curve is non-monotone, its apparent recovery above $\varphi = 0.33$ being the safety failure of Figure 4.18, not restored liveness.

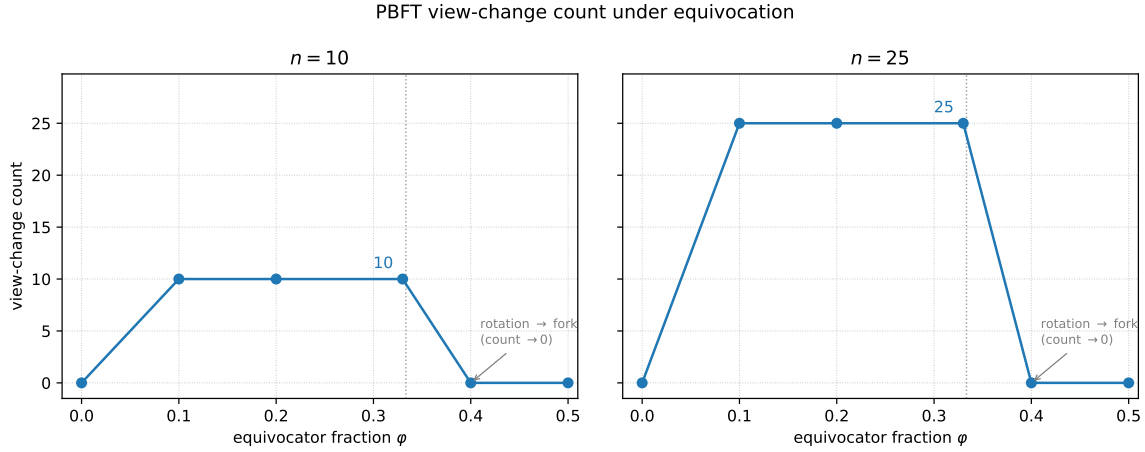


Figure 4.17: PBFT view-change count under equivocation. Number of view-changes against the equivocator fraction φ , on a count axis shared across the two committee sizes so the size effect is visible; the count rises to its plateau — ten at $n = 10$ and twenty-five at $n = 25$ — at the last safe fraction and collapses to zero at $\varphi = 0.40$, where the deterministic fork replaces leader rotation.

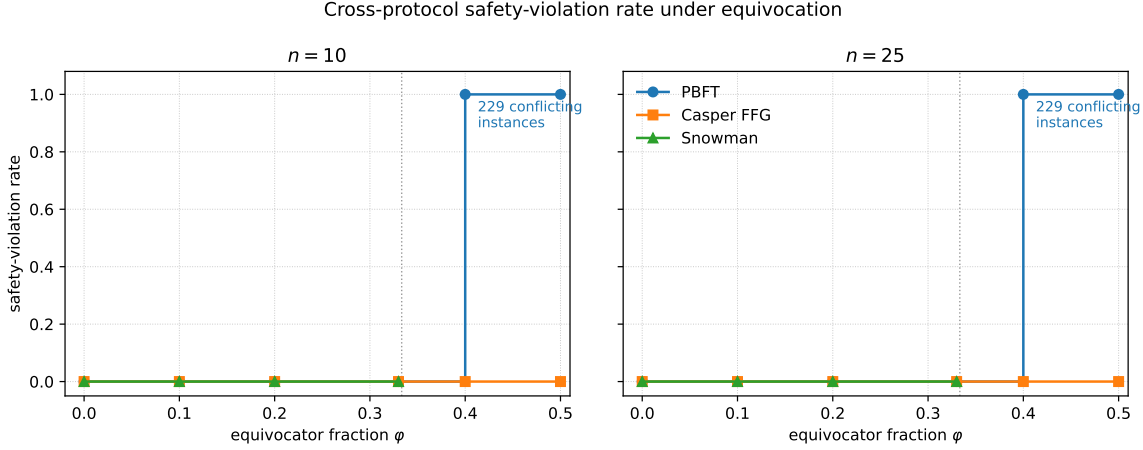


Figure 4.18: Cross-protocol safety-violation rate under equivocation. Safety-violation rate against the equivocator fraction ϕ , drawn as steps, one curve per protocol and faceted by validator count; only PBFT departs from zero, stepping to a deterministic fork at $\phi = 0.40$, where the magnitude of the fork — 229 conflicting (`view`, `seq`) instances at both committee sizes — is annotated. Casper FFG and Snowman stay at zero; their failure modes appear on the stake axis of Figure 4.19 and through ε respectively.

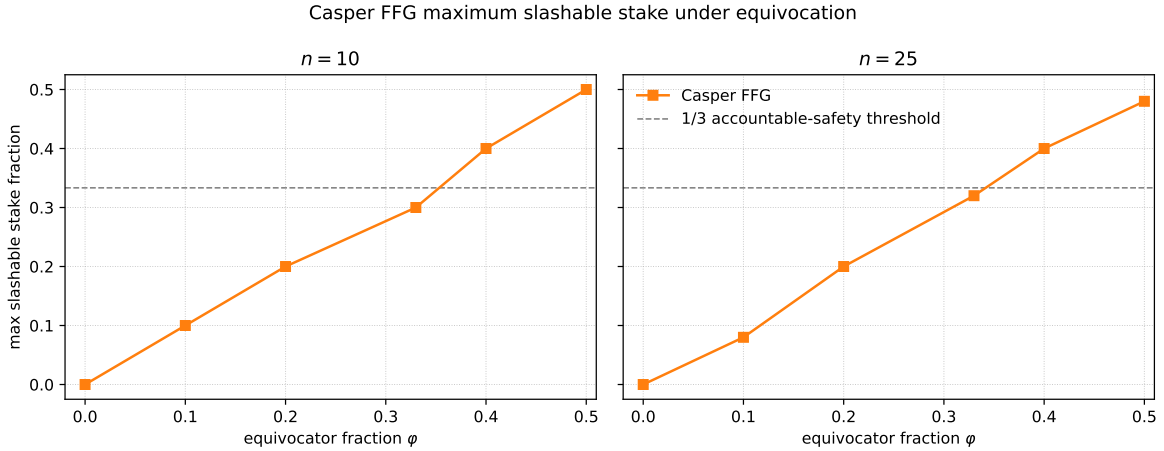


Figure 4.19: Casper FFG slashable stake under equivocation. Maximum slashable stake fraction against the equivocator fraction ϕ , faceted by validator count, with the one-third accountability line marked.

4.4.4 The performance–security tradeoff

The three strategies together answer RQ4. No protocol is robust to every adversary: the structural choice that defends a protocol against one strategy is the same choice that exposes it to another (Table 4.2, rendered as an outcome map in Figure 4.21). PBFT’s exact quorum and view-change recovery make it the strongest protocol against the two liveness adversaries — immune to delayed voting and undegraded under silence up to its quorum cliff — but

the same leader-based, non-accountable commit rule makes it the worst under equivocation, the only protocol whose safety failure is both deterministic and unattributable. Snowman’s subsampling inverts this profile: it is the strongest against equivocation, presenting no fork surface and a vanishing probabilistic bound, but it is the most fragile to silence, its polls starving earliest, and the costliest under delay, its time-to-finality blowing up by factors of 62 and 49. Casper FFG wins no single adversary outright, yet it is the only protocol with an accountable safety failure and the most liveness-robust to equivocation, while paying for its single rotating proposer with the earliest liveness loss under delay and a graceful but real throughput decay under silence. The protocol best on one axis is last on another in every case: PBFT first against delay and silence but last against equivocation, Snowman first against equivocation but last against silence, Casper FFG never first but never catastrophic.

Table 4.2: **Adversarial outcomes by protocol and strategy** ($n = 10/25$, **20 seeds**). Values pair the two committee sizes where they differ. Robustness order is per strategy, ranked on the liveness held for the two liveness adversaries and on the safety invariant for equivocation.

Adversarial strategy	PBFT	Casper FFG	Snowman	Robustness order
Delayed voting	immune; finality $1.0\times$, no view-changes	liveness dips (success $\rightarrow 0.60 / 0.65$)	survives; finality $\times 62 / \times 49$	PBFT $\approx$ Snowman $\gg$ FFG
Silent non-participation	clean quorum cliff at $\varphi = 0.40$; no decay below it	graceful decay, survives to $\varphi = 0.33$ (throughput $\approx 1 - \varphi$)	early cliff at $\varphi = 0.10/0.20$; starves	PBFT $\approx$ FFG > Snowman
Equivocation	deterministic unaccountable fork at $\varphi = 0.40$ (229 conflicts)	accountable: $\geq \frac{1}{3}$ stake slashable at $\varphi = 0.40$, no fork	no fork surface; $\varepsilon \approx 5 \times 10^{-15} / 3 \times 10^{-11}$	Snowman > FFG > PBFT

This no-dominance result is not an artifact of how the adversaries were chosen. The three strategies are the generic capabilities of the adversary catalog, defined independently of any protocol rather than reverse-engineered to strike each protocol’s known weakness, and every protocol is exposed to all three. The contribution of the section is therefore not the bare statement that no protocol wins everywhere — which a reader of the design space might anticipate — but the mechanism-level mapping of which structural feature produces which failure under which adversary, and the inversions that mapping reveals: that the same subsampling which makes Snowman the most delay-tolerant protocol when peers are merely slow makes it the least tolerant when they fall silent, and that PBFT’s leader-based commit rule is at once the source of its liveness robustness and of its unaccountable fork.

Two qualifications bound this verdict. First, the adversarial verdict is scoped to the three families evaluated. The swept strategies are the three generic capabilities of the catalog; the leader-disruption surface, plausibly the sharpest attack on the leader-based protocols, is catalogued but not exercised, and because the delayed and silent sets are chosen to spare the view-0 primary, PBFT’s standing as the strongest protocol against the two liveness adversaries holds only against adversaries that leave its leader honest — a leader-targeting adversary

is precisely the case this sweep does not measure. Second, several measurement boundaries qualify how the numbers read. Safety results are seed-invariant, since the equivocating set is fixed rather than sampled, so the safety columns carry no seed variance while the success-rate columns carry all of it. Snowman’s analytical ε is not empirically witnessed at the baseline confidence depth, as noted in §4.4.3. A run truncated at the measurement deadline is not scored a liveness failure, only a run in which no honest validator commits within the window (§3.4.1). And the latency-only network charges no signature-verification or other compute cost, so the work of detecting and recovering from equivocation — view-change rounds for PBFT, slashing-evidence processing for Casper FFG — is counted in messages but not in computation. This omission understates the cost borne by precisely PBFT and Casper FFG, whose equivocation handling is compute-bound, and so flatters those two on any cost comparison; it does not bear on the message-count, liveness, or safety verdicts this section draws (§3.2).

Read on the throughput axis rather than the liveness one, the same sweep answers RQ2: as the injected Byzantine fraction φ rises toward the fault threshold, sustained throughput degrades in three distinct modes — PBFT holds undegraded until its quorum cliff, Casper FFG decays gracefully in proportion to the participating stake ($\approx 1 - \varphi$), and Snowman starves earliest as its polls fail to close — so the rate at which throughput falls is governed by each family’s quorum structure rather than by φ alone (Figure 4.20).

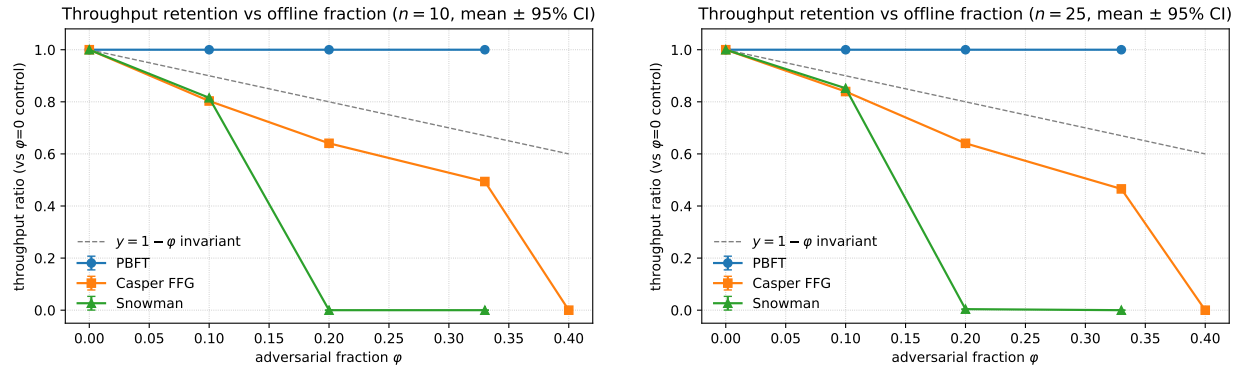


Figure 4.20: Throughput degradation versus adversarial fraction (silent non-participation). Committed-unit throughput against the injected silent fraction φ for each protocol, one panel per committee size ($n = 10$, left; $n = 25$, right); the three RQ2 degradation modes read off the curves directly — PBFT undegraded to its quorum cliff, Casper FFG decaying in proportion to the participating stake ($\approx 1 - \varphi$), and Snowman starving earliest.

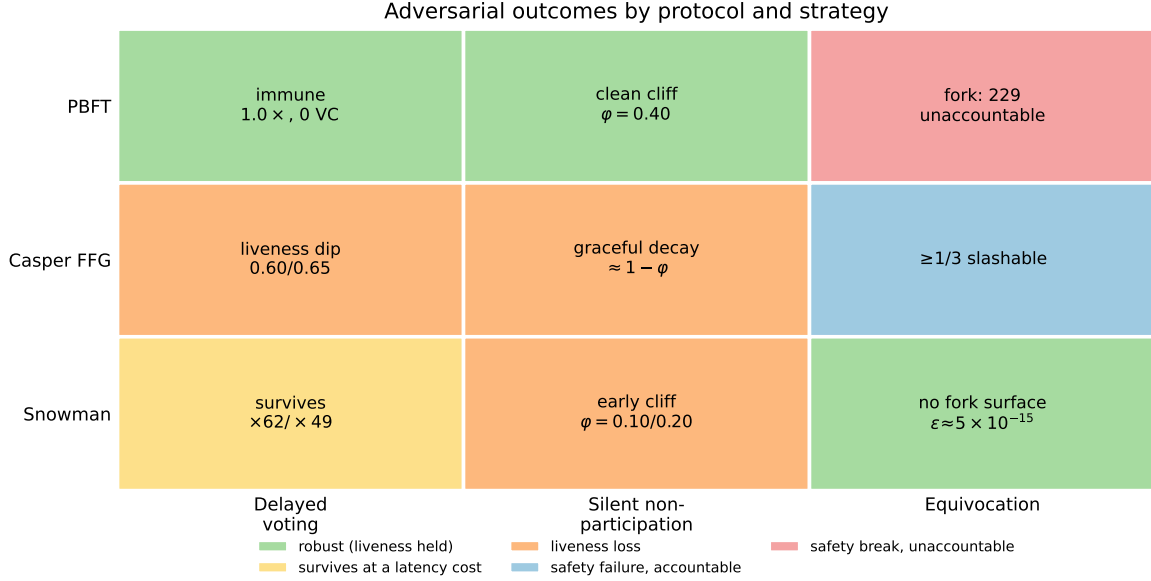


Figure 4.21: Adversarial outcomes by protocol and strategy. The nine protocol–strategy cells of Table 4.2 rendered as an outcome map: the cell color encodes the kind of outcome — robust with liveness held, survival at a latency cost, liveness loss, accountable safety failure, or unaccountable safety break — and each cell label carries the governing magnitude. No protocol occupies a single color across its row: PBFT runs from robust under the two liveness adversaries to an unaccountable break under equivocation, Snowman from costly survival under delay through liveness loss under silence to robust under equivocation, and Casper FFG is never first yet never catastrophic. The image carries the no-dominance verdict and the structural inversions that produce it.

The question of whether any one family occupies a dominant position once the baseline, delay, and adversarial regimes are considered jointly — the Pareto-frontier synthesis of RQ5 — is taken up in Chapter 5.

Chapter 5

Synthesis

5.1 Chapter roadmap

Chapter 4 closed on a question it had assembled but deliberately did not answer. Each of its three movements isolated one stress axis and reported how the protocols behaved under it: validator-set size in the baseline, network delay in the second sweep, and adversarial behavior in the third. Read separately, each sweep produced a clear per-axis verdict. Read together, they raise the question this chapter takes up: whether any one family occupies a dominant position once the baseline, delay, and adversarial regimes are considered jointly. This is the Pareto-frontier synthesis of RQ5.

RQ5 asks whether a consistent Pareto frontier of the performance–security tradeoff exists across the families and whether any family dominates the others across all operating regimes. The frontier reported here is traced over the three protocols evaluated throughout this study: PBFT, Casper FFG, and Snowman. This chapter introduces no new measurements; it collates the per-axis results of Chapter 4 into a single comparison and reads the shape of the tradeoff off them.

The answer is that no family dominates. Each of the three protocols is the strict best on at least one axis that no other matches, so each is a non-dominated point on the frontier; and the frontier itself has a definite shape: no configuration is at once cheap, fast, and resilient. The contribution of the synthesis is therefore not the bare no-dominance verdict, which a reader of the design space might anticipate, but the map of which structural choice places each family where on the frontier, and the inversions that map reveals.

5.2 The comparison and its axes

A synthesis over heterogeneous sweeps requires a common frame. The frame used here is the performance–security plane: each protocol is characterized by its position on a set of axes drawn from the three sweeps, and one family is said to dominate another when it is no worse on every axis and strictly better on at least one. A family that no other dominates lies on the Pareto frontier. The axes are the quantities Chapter 4 already reports: baseline commit latency and communication overhead from the scaling sweep, time-to-finality under network delay and finalization under packet loss from the delay sweep, and liveness and safety

under each adversarial strategy from the adversarial sweep. The synthesis adds a reading, not a measurement. These axes are the primary metrics of the four data-generating research questions rather than a set chosen to make the families differ, so a family that dominated would do so on the very quantities the evaluation was designed to measure.

Two conventions fixed in Chapter 3 and carried through Chapter 4 govern the comparison and are restated because the reading rests on them. Cross-protocol latency is read from `commit_latency_ms`, the canonical time-to-finality column, so that the three protocols’ irreversibility milestones are compared like for like; cross-protocol throughput is read from `goodput`, the rate of committed transaction bytes, rather than from the protocol-granularity decision-event rate. Every comparative verdict below inherits one further qualification: the simulator charges network latency but no signature-verification or other compute cost, so cost and per-validator comparisons flatter the protocols whose equivocation handling is compute-bound, namely PBFT and Casper FFG, while the message-count, liveness, and safety verdicts are unaffected.

5.3 Where each family sits on the frontier

5.3.1 PBFT: fast and live, but the fork is unaccountable

PBFT occupies the low-latency, high-liveness corner of the frontier. At the honest baseline it commits in approximately one second, level with Snowman and about five times faster than Casper FFG’s epoch-granularity finality of roughly five seconds, and its latency is flat in the validator set. Its communication overhead grows linearly per committed unit, at approximately $2n$ messages, the atomic-commit-unit denominator absorbing one factor of the all-to-all $O(n^2)$ instance cost. Under network delay it is the least exposed of the three, adding about nine-tenths of a second under moderate uniform delay where Snowman pays an order of magnitude. It is the most loss-resilient protocol, the only one still finalizing at twenty-percent packet loss, ranked first by area under the finalization-rate curve at $n = 10$ and tied for first at $n = 25$. Against the two liveness adversaries it is the strongest of the three. It is immune to delayed voting at unit finality cost, undegraded under silent non-participation through $\varphi = 0.33$, and collapses only at $\varphi = 0.40$, the first sampled fraction past its $2f + 1$ quorum bound. These liveness verdicts are established against adversaries that leave the view-0 primary honest; the leader-disruption surface, plausibly the sharpest attack on a leader-based protocol, is catalogued but not measured, so PBFT’s liveness standing is bounded to non-leader-targeting strategies.

The mechanism that buys this liveness is the one that disqualifies PBFT on the axis it loses. Its leader-based, exact-quorum commit rule recovers from a stalled or slow leader by view-change rotation, which is why it survives delay, loss, and silence. That same commit rule is non-accountable. Under equivocation past the fault threshold it produces a deterministic fork — 229 conflicting committed instances at $\varphi = 0.40$ — with no slashable evidence identifying the equivocators. View-change activity makes the double edge visible: the mechanism fires for successful recovery at fractions up to one-third and degenerates into thrashing above it. PBFT is thus the performance and liveness leader of the three. It is also the only family whose safety failure is both certain above the threshold and unattributable [16].

5.3.2 Casper FFG: never fastest, but the only accountable failure

Casper FFG wins no latency or resilience axis outright, yet it is non-dominated on two counts. The first is cost: per committed unit it is the cheapest of the three, at approximately $1.2n$ messages against PBFT’s $2n$ and far below Snowman’s polling overhead. On the latency and resilience axes, by contrast, it trails. It is the slowest of the three at the baseline, finalizing at epoch granularity in roughly five seconds, and the lowest in honest goodput. It is the least loss-resilient, ranked last by area under the finalization-rate curve at both committee sizes and the first to fall silent under packet loss. Under silence it degrades gracefully, its sustained throughput falling in proportion to the participating stake, approximately $1 - \varphi$, and surviving to $\varphi = 0.33$.

The second count appears on the equivocation axis. Where PBFT forks without attribution, Casper FFG’s slashing conditions convert an equivocation into accountable evidence: at $\varphi = 0.40$ the protocol holds with no in-model fork while exposing a slashable stake fraction at or above one-third. No other family in the study offers an accountable safety failure. Casper FFG occupies that corner of the frontier alone [18]. The cost it pays is concentrated on liveness under loss, and that cost is the measured analogue of the failure that motivated this study. An epoch whose attestations fail to reach a quorum leaves finality stalled. The quorum is missed by a lossy network here, and by delay under attestation-processing pressure in Ethereum’s multi-epoch finality stall of May 2023 [7] there. The simulated failure is the same class observed in that deployment.

5.3.3 Snowman: safest under equivocation, costliest under delay

Snowman occupies a corner defined by a single mechanism pulling in opposite directions on different axes. Its K -peer subsampling gives it the strongest safety posture against equivocation of the three: it presents no fork surface, and its probabilistic safety bound is vanishing, with an analytical ε near 5×10^{-15} at $n = 10$. That bound is reported rather than empirically witnessed; an empirical fork count of zero at the baseline confidence depth is a non-witness of the bound, not a confirmation of it. On the light-loss plateau at $n = 25$ it retains the highest finalization rate of the three — 0.904 at five-percent loss against PBFT’s 0.533 — the redundancy of its polling scaling with the committee.

The same subsampling makes it the costliest protocol under delay and the most fragile under silence. Acceptance requires β sequential polling rounds, and each round waits on the slowest of K sampled peers, so delay compounds. Under moderate uniform delay Snowman’s time-to-finality grows roughly twelve- to fifteenfold, against a fraction of that for the other two, and under a delayed-voting adversary its finality cost reaches a factor of sixty-two. When peers fall silent rather than merely slow, the polls starve. Snowman cliffs earliest of the three: it finalizes under a silent tenth of the validators but starves once the silent fraction reaches $\varphi = 0.20$ at $n = 10$, below the one-third the other two tolerate. The cliff is committee-size dependent. The larger $n = 25$ sample absorbs more non-responders per poll and defers the starvation point to $\varphi = 0.33$, yet Snowman still fails one sampled step before the quorum protocols, remaining the earliest of the three to lose liveness under silence. The inversion is the sharpest single result of the study — the identical structural choice makes Snowman the most delay-tolerant family when peers are merely slow and the least tolerant when they go

silent [11].

5.4 The frontier and the no-dominance verdict

Collecting the per-family positions gives the frontier its shape (Table 5.1, visualized as the overlaid radar of Figure 5.1). No row of the table is won by a single family across the board, and three families each win a row no other does: PBFT the delay, loss, and liveness axes; Casper FFG the communication-overhead and accountability axes; Snowman the equivocation-safety axis. Each is therefore non-dominated, and no family dominates — the direct answer to RQ5 over the three protocols evaluated. This verdict does not rest on the one row only a slashing-based protocol can win: setting the accountable-safety axis aside, each family is still non-dominated on a measured axis — Casper FFG on communication overhead, Snowman on equivocation safety, and PBFT on delay, loss, and liveness — so the multi-cornered shape survives the removal of the definitional row.

Table 5.1: **Cross-regime comparison of the three families on the performance–security plane** ($n = 10/25$, **20 seeds; 8 for the Snowman $n = 25$ delay cell**). Each row is one axis from the Chapter 4 sweeps; the final column names the family or families that are strict best on that axis. No family wins every row, and each of the three wins at least one row no other does, so each is non-dominated. Two rows are not symmetric contests: the equivocation-safety row ranks Snowman first on its analytical bound ε , reported rather than empirically witnessed (§5.3.3), and the accountable-safety row names a capability only a slashing-based protocol can offer, so Casper FFG holds it uncontested by construction rather than by winning a comparison. The loss-resilience row reports the $n = 10$ ranking, where PBFT leads cleanly; at $n = 25$ PBFT and Snowman are a statistical tie at the top, their area-under-the-retention-curve confidence intervals overlapping (Snowman 0.369 [0.366, 0.372], PBFT 0.351 [0.327, 0.376]) on a reduced Snowman seed count. Values pair the committee sizes only where they differ; full per- n figures are in the cited pages.

Axis (regime)	PBFT	Casper FFG	Snowman	Best
Baseline commit latency	≈ 1 s	≈ 5 s	≈ 1 s	PBFT $\approx$ Snowman
Communication overhead per unit	$\approx 2n$	$\approx 1.2n$	$\approx 2K\beta$ ($\approx 14\times$ PBFT at $n = 16$)	Casper FFG
Time-to-finality under delay	$+ \approx 0.9$ s	$+ \approx 27\%$	$\times 12\text{--}13$	PBFT
Loss resilience (AURC; survival depth)	first; alive at 20% loss	last	AURC tie at $n = 25$; cliffs by 10% loss	PBFT
Liveness under delayed voting	immune ($1.0\times$)	dips (success $\rightarrow 0.60$)	survives at $\times 62$ finality	PBFT
Liveness under silence	clean to $\varphi = 0.33$, cliff at $\varphi = 0.40$	graceful to $\varphi = 0.33$	cliff at $\varphi = 0.20$ ($n = 10$) / $\varphi = 0.33$ ($n = 25$)	PBFT $\approx$ FFG
Safety under equivocation	deterministic fork at $\varphi = 0.40$	accountable, no fork	no fork surface; $\varepsilon \approx 5 \times 10^{-15}$ / 3×10^{-11}	Snowman
Accountable safety	none (unattributable fork)	slashable $\geq \frac{1}{3}$ stake	not applicable (probabilistic)	Casper FFG

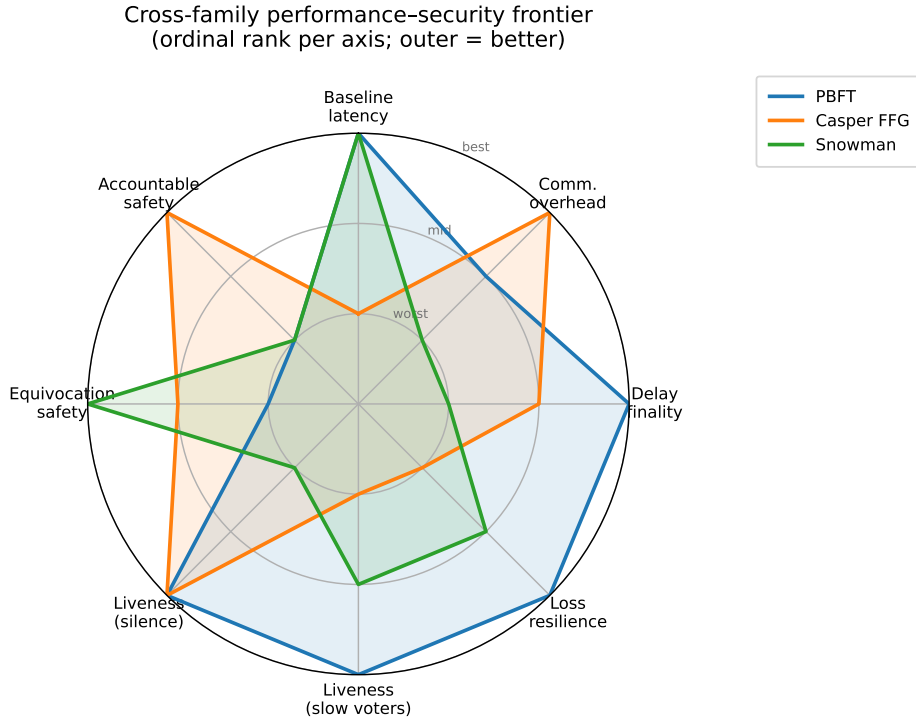


Figure 5.1: Cross-family performance–security frontier. The three families evaluated, scored on the eight cross-regime axes of Table 5.1 and normalized by ordinal rank per axis: the outer ring marks the strict best on an axis and the center the worst, with ties shared. The polygons overlap and none encloses another. Each family reaches the outer ring on at least one axis no other matches — PBFT on the delay, loss, and liveness axes, Casper FFG on communication overhead and accountable safety, Snowman on equivocation safety — so each is non-dominated and no family dominates, the verdict of Table 5.1 read directly off one image. The rank scale sets aside the magnitudes that drive it, which are the Chapter 4 headlines: Snowman’s time-to-finality grows roughly sixty-twofold under delayed voting and its polling overhead reaches about fourteen times PBFT’s, PBFT forks into 229 conflicting committed instances past its threshold, and Snowman’s analytical safety bound is near 5×10^{-15} while Casper FFG exposes at least one-third of stake as slashable. Two axes are not symmetric contests, as in Table 5.1: equivocation safety ranks Snowman first on its reported analytical bound rather than an empirical witness, and accountable safety names a capability only a slashing-based protocol offers.

Two features of the frontier carry more weight than the bare verdict. The first is a gap in it rather than a point on it. Resilience under loss is bought with latency. The operator

tradeoff of Figure 4.13 shows that the protocols which retain finalization under loss are exactly the ones that pay the most latency to do so. PBFT and Snowman inflate time-to-finality by factors of 2.16 (Snowman, $n = 10$) to 3.57 (PBFT, $n = 25$). Each factor is measured at that protocol’s own deepest surviving loss level rather than a common one, and over the seeds that still finalize there, so the two figures are not read off a single point of Figure 4.13. Casper FFG refuses that trade and stays near unit latency, and dies first. The cheap-and-resilient corner of the plane is empty: resilience is bought with latency, and no measured configuration escapes the purchase. The second is that the rankings invert across axes. The family first against delay and silence is last against equivocation, the family first against equivocation is last against silence, and the family never first is never catastrophic either. The frontier is consistent, holding a stable shape across the regimes, but it is genuinely multi-cornered, and a deployment’s position on it is fixed by which threat it must tolerate rather than by a single best protocol.

5.5 Implications and hand-off

The practical content of the no-dominance result is the mapping itself. Because each structural choice that defends a family on one axis exposes it on another, protocol selection cannot be reduced to a single ranking and must instead be read against the deployment’s dominant threat: a system that above all must not fork without attribution is served by Casper FFG’s accountable failure; one that must hold liveness through network turbulence by PBFT’s view-change recovery; one that must resist equivocation outright by Snowman’s subsampling. Each of these choices accepts the cost the same mechanism imposes elsewhere. The deployment incidents that opened this study (§1.2) are heterogeneous in exactly this way: a liveness halt under load on one network and a multi-epoch finality stall on another are different failure classes, and the synthesis here is that they are not competing symptoms of one immature technology but the predictable shadows of distinct structural commitments.

This chapter answered the last of the five research questions and, with it, closed the comparative arc that began with the design space of Chapter 2. What remains is to draw the individual answers together, to state plainly the boundaries within which they hold, and to identify the directions the evaluation leaves open — the work of Chapter 6.

Chapter 6

Conclusions

6.1 Summary of findings

This thesis set out to measure, on matched assumptions and in a single harness, how representative Layer-1 consensus protocols behave under the network and adversarial conditions their guarantees assume away. The simulator, its metric schema, and its experiment matrix were built in Chapter 3; the campaign ran in Chapter 4 and was synthesized in Chapter 5. The five research questions are now answered over the three protocols evaluated — PBFT, Casper FFG, and Snowman. The five answers are collected in Table 6.1 and developed below.

Table 6.1: **The five research questions and their answers over the three protocols evaluated.**

RQ	Question	Answer	Governing mechanism
RQ1	latency under rising network-delay variance	flat in n ; PBFT + ≈ 0.9 s, Casper FFG + $\approx 27\%$, Snowman $\times 12$ –13	round structure vs. β sequential polls
RQ2	sustained throughput as φ rises to the threshold	three modes: PBFT holds then cliffs, Casper FFG $\approx 1 - \varphi$, Snowman starves earliest	quorum structure
RQ3	communication overhead per committed unit	PBFT $\approx 2n$, Casper FFG $\approx 1.2n$ (cheapest), Snowman $\approx 2K\beta$ ($\approx 14\times$ PBFT at $n = 16$)	all-to-all / attestation vs. K -poll
RQ4	which adversary causes liveness or safety loss	no protocol robust to all three; the mechanism map	each structural defense is also an exposure
RQ5	consistent Pareto frontier; any dominance	a frontier exists; no family dominates	each family non-dominated on ≥ 1 axis

RQ1 asked how commit latency scales as network-delay variance rises. The three families carry delay very differently: PBFT adds under a second under moderate delay, Casper FFG

roughly twenty-seven percent — an increase dominated by its slot-clock rescaling rather than by attestation propagation — and Snowman an order of magnitude, the last being the only protocol sensitive to the shape of the delay distribution. Commit latency is otherwise flat in the validator set. The network timeline, not the committee size, governs time-to-finality. RQ2 asked how sustained throughput degrades as the Byzantine fraction approaches the fault threshold. Throughput degrades in three distinct modes — PBFT undegraded until a hard quorum cliff, Casper FFG decaying in proportion to the participating stake at approximately $1 - \varphi$, and Snowman starving earliest — so the rate of throughput loss is set by each family’s quorum structure rather than by the fault fraction alone. RQ3 asked after relative communication overhead. Per committed unit, PBFT and Casper FFG grow linearly in the validator set, while Snowman’s subsampled polling costs an order of magnitude more at thesis scale, roughly fourteenfold at $n = 16$.

RQ4 asked which adversary drives which protocol to a liveness loss, a safety violation, or neither. No protocol is robust to every adversary. The structural choice that defends a family against one strategy is the same that exposes it to another, and the contribution is the resulting mechanism map. PBFT is immune to the liveness adversaries exercised here — those that spare the view-0 primary — but is the source of the only unaccountable fork. Snowman is the equivocation-safety leader on its analytical bound, reported rather than empirically witnessed, yet the most delay- and silence-exposed of the three. Casper FFG is never first against any single adversary but holds the only accountable failure. RQ5 asked whether a consistent performance–security frontier exists and whether any family dominates. A consistent frontier exists across the three families. No family dominates: each is the strict best on at least one axis, and the frontier admits no configuration that is at once cheap, fast, and resilient. The verdict does not turn on the one axis only a slashing-based protocol can hold: even setting accountable safety aside, each family remains non-dominated on a measured axis.

Returning to the incidents that motivated the study (§1.2), the measured failure modes are the controlled analogues of the deployment ones. An attestation quorum lost to a lossy network reproduces the class of finality stall observed on Ethereum in May 2023 [7], and a leader-based protocol’s view-change behavior under stress reproduces the class of liveness halt observed when block production stalls. The synthesis is that these are not symptoms of one immature technology but the consequences of distinct structural commitments, which is why no single protocol resolves all of them.

6.2 Limitations

The findings hold within boundaries that Chapter 3 fixed and the analysis honored throughout.

The cost model. The simulator charges network latency but no signature-verification, execution, or bandwidth cost. This boundary bears on cost and per-validator verdicts, which it flatters for the compute-bound equivocation handling of PBFT and Casper FFG. It does not bear on the message-count, liveness, or safety results. The throughput model has no saturation ceiling: goodput is reported against an offered load below any saturation point, so the flat-in- n goodput is a property of the unsaturated model rather than a claim about peak capacity.

Commensurability. Thesis-scale committee sizes require rescaling protocol parameters (Snowman’s K , α_c , and β , and the Casper FFG slot-to-delay coherence rule), so the cross-protocol verdicts rest on those conventions. They are reported as robust only where they survive the governing sensitivity check. The evaluation covers the three protocols implemented in the harness, and the comparative verdicts are scoped to them. The frontier is likewise traced only over the regimes measured; a high-throughput regime outside that span is not represented, so the absence of a configuration that is at once cheap, fast, and resilient is a statement about the measured plane rather than the whole design space. Snowman’s safety is reported through its analytical bound $\varepsilon \leq (1 - \alpha_c/K)^\beta$ rather than a fork count, and an empirical zero is a non-witness of that bound, not a confirmation of it. Several rankings also rest on narrow support: the loss-resilience comparison of the two most resilient families at $n = 25$ is a statistical tie, their confidence intervals overlapping on a reduced seed count (Chapter 5, Table 5.1), so the absence of a dominant family there is a non-rejection rather than a measured separation.

The adversarial sweep. Packet loss is modeled as permanent per-message drop with no transport retransmission, so the loss-resilience curves are an upper bound on fragility rather than a model of a retransmitting transport. The sweep exercises the three generic capabilities of the adversary catalog and spares the view-0 primary, so the leader-disruption surface is catalogued but not measured. That surface is plausibly the sharpest attack on the leader-based protocols. PBFT’s liveness standing is therefore established only against adversaries that leave its leader honest.

6.3 Directions for further work

6.3.1 Production-optimized protocol variants

The communication-overhead comparison of §4.2.4 evaluates each protocol at the message granularity of its original specification: classical PBFT with all-to-all prepare and commit phases [16], and Casper FFG with individually signed attestations counted toward a super-majority [18]. Production deployments of both families reduce this cost through signature aggregation. The Ethereum beacon chain aggregates committee attestations with BLS signatures, which collapses Casper FFG’s per-epoch attestation cost from the $O(n^2)$ of propagated individual votes to $O(n)$ [5]; HotStuff achieves the analogous reduction for the PBFT family, replacing the quadratic vote phases with threshold-signature collection at the leader to obtain linear normal-case and view-change communication [17]. Modeling these aggregated variants would establish whether the per-unit cost ordering observed at the as-specified granularity survives at the granularity of deployed systems.

This extension carries a methodological requirement that constrains how it must be undertaken. Signature aggregation is a property of a protocol family’s signature scheme rather than of an individual protocol; introducing it for one family while leaving another at its un-aggregated specification would compare implementations at different levels of optimization and would therefore misstate the per-unit cost contrast that answers RQ3. A faithful extension consequently either models all signature-based families at their production-optimized message granularity — aggregated Casper FFG against a HotStuff-style PBFT — or reports the

as-specified and the aggregated regimes side by side, so that the level of optimization is held constant across the comparison. Of the two, side-by-side reporting is the more conservative: it preserves the present as-specified baseline as a point of comparison rather than replacing it, so the per-unit ordering reported here remains legible alongside the optimized one. The final choice between side-by-side reporting and a uniformly aggregated comparison is a methodological decision left to the supervisor. An implementation plan for the Casper FFG side — the aggregation topology, the message-counting convention, and the comparability decision described here — is recorded as a kickoff specification in the project repository.

6.3.2 Further directions

Beyond the production-optimized variants of §6.3.1, the evaluation leaves several directions open. The most direct is a saturation-throughput capacity model: the present baseline reports goodput against an offered load below saturation, so throughput reads as flat in the validator set, and replacing the unsaturated model with one that drives each protocol to a measured ceiling would turn the flat-goodput baseline into a peak-capacity comparison. A second is an adaptive-timeout enhancement: exponential backoff with jitter, calibrated to observed round-trip time, which the simulator is positioned to evaluate against the baseline. A meaningful comparison would require a regime that stresses timeout calibration directly, such as a tight view-change budget under high-jitter delay or a post-GST recovery scenario. The steady-state sweeps reported here are not such a regime: in them the view-change budget was set generously enough that recovery timers seldom fired. A third is to witness Snowman’s analytical safety bound empirically by driving the protocol at a weakened confidence depth where forks become observable, closing the gap between the reported bound and a measured rate. A fourth is to extend the harness to a DAG-based family (Narwhal+Tusk), whose data-availability-withholding adversary the present sweep does not cover; adding it would populate the high-throughput corner that the three-family frontier leaves unmeasured. Finally, the campaign runs at thesis-scale committee sizes under a latency-only network; repeating the comparison at larger validator sets and against a transport that models bandwidth and retransmission would test how far the rankings reported here survive outside the simplifying assumptions that made the controlled comparison possible.

6.4 Concluding remarks

The contribution of this thesis is a single harness in which representative Layer-1 consensus protocols could be subjected to the same delay and adversarial conditions and measured against one schema. Across the three protocols evaluated, the comparative reading that harness made possible has a single headline: not a winner, but a map. No family dominates. The result is an account of which structural commitment places each family where on the performance–security frontier.

The same structural choice that places a family on one corner is what exposes it on another, so the result is a map of mechanisms rather than an artifact of the comparison set. The deployment failures that opened the study motivated the questions the harness answers. Liveness halts and finality stalls are not interchangeable faults to be engineered away by a

single better protocol. They are the separable consequences of the choices a protocol makes, consequences that an evaluation on matched assumptions can name.

Appendix A

Code listing

This example uses the listings package.

```
1 \begin{luacode*}
2 function print_rate(kappa,xMin,xMax,npoints,option)
3     local c = 1-kappa*kappa
4     local croot = (1-kappa*kappa)^(1/2)
5     local logx = math.log(xMin)
6     local psi = 0
7
8     local xstep = (math.log(xMax)-math.log(xMin))/(npoints-1)
9
10    arg0 = math.sqrt(xMin/c)
11    psi0 = (1/c)*math.exp((kappa*arg0)^2)*(erfc(kappa*arg0)-erfc(
        arg0))
12
13    if option~=[[[]]] then
14        tex.sprint("\addplot+[\"..option..\" coordinates{")
15        -- addplot+ for color cycle to work
16    else
17        tex.sprint("\addplot+ coordinates{")
18    end
19    tex.sprint("("..xMin..","..psi0..")")
20
21    for i=1, (npoints-1) do
22        x = math.exp(logx + xstep)
23        arg = math.sqrt(x/c)
24        karg = kappa*arg
25        if karg<5 then
26            -- this break compensates for exp(karg^2), which multiplies the
                error in the erf approximation...
27            logpsi = -math.log(croot) + karg^2 + math.log(erfc(karg)-
                erfc(arg))
28            psi = math.exp(logpsi)
29        else
```

```

30      psi = (1/(karg) - 1/(2*(karg^3)) + 3/(4*(arg^5)) )/(1
          .77245385*croot)
31      -- this is the large x asymptote of the reaction rate
32      end
33      logx = math.log(x)
34      tex.sprint("(" .. x .. ", " .. psi .. ")")
35      end
36      tex.sprint("}")
37 end
38 \end{luacode*}

```

Appendix B

One-term coefficients for heat conduction

B.1 A multipage table of numbers

This example uses the `longtable` package: $\theta = A_1 f_1 \exp(-\lambda_1^2 \text{Fo})$, $\bar{\theta} = D_1 \exp(-\lambda_1^2 \text{Fo})$.

Table B.1: One-term coefficients for one-dimensional heat conduction with a convective boundary condition. Data follow H. D. Baehr and K. Stephan [baehr1998].

Bi	<i>Plate</i>			<i>Cylinder</i>			<i>Sphere</i>		
	λ_1	A_1	D_1	λ_1	A_1	D_1	λ_1	A_1	D_1
0.01	0.09983	1.0017	1.0000	0.14124	1.0025	1.0000	0.17303	1.0030	1.0000
0.02	0.14095	1.0033	1.0000	0.19950	1.0050	1.0000	0.24446	1.0060	1.0000
0.03	0.17234	1.0049	1.0000	0.24403	1.0075	1.0000	0.29910	1.0090	1.0000
0.04	0.19868	1.0066	1.0000	0.28143	1.0099	1.0000	0.34503	1.0120	1.0000
0.05	0.22176	1.0082	0.9999	0.31426	1.0124	0.9999	0.38537	1.0150	1.0000
0.06	0.24253	1.0098	0.9999	0.34383	1.0148	0.9999	0.42173	1.0179	0.9999
0.07	0.26153	1.0114	0.9999	0.37092	1.0173	0.9999	0.45506	1.0209	0.9999
0.08	0.27913	1.0130	0.9999	0.39603	1.0197	0.9999	0.48600	1.0239	0.9999
0.09	0.29557	1.0145	0.9998	0.41954	1.0222	0.9998	0.51497	1.0268	0.9999
0.10	0.31105	1.0161	0.9998	0.44168	1.0246	0.9998	0.54228	1.0298	0.9998
0.15	0.37788	1.0237	0.9995	0.53761	1.0365	0.9995	0.66086	1.0445	0.9996
0.20	0.43284	1.0311	0.9992	0.61697	1.0483	0.9992	0.75931	1.0592	0.9993
0.25	0.48009	1.0382	0.9988	0.68559	1.0598	0.9988	0.84473	1.0737	0.9990
0.30	0.52179	1.0450	0.9983	0.74646	1.0712	0.9983	0.92079	1.0880	0.9985
0.40	0.59324	1.0580	0.9971	0.85158	1.0931	0.9970	1.05279	1.1164	0.9974
0.50	0.65327	1.0701	0.9956	0.94077	1.1143	0.9954	1.16556	1.1441	0.9960
0.60	0.70507	1.0814	0.9940	1.01844	1.1345	0.9936	1.26440	1.1713	0.9944
0.70	0.75056	1.0918	0.9922	1.08725	1.1539	0.9916	1.35252	1.1978	0.9925
0.80	0.79103	1.1016	0.9903	1.14897	1.1724	0.9893	1.43203	1.2236	0.9904
0.90	0.82740	1.1107	0.9882	1.20484	1.1902	0.9869	1.50442	1.2488	0.9880
1.00	0.86033	1.1191	0.9861	1.25578	1.2071	0.9843	1.57080	1.2732	0.9855
1.10	0.89035	1.1270	0.9839	1.30251	1.2232	0.9815	1.63199	1.2970	0.9828
1.20	0.91785	1.1344	0.9817	1.34558	1.2387	0.9787	1.68868	1.3201	0.9800
1.30	0.94316	1.1412	0.9794	1.38543	1.2533	0.9757	1.74140	1.3424	0.9770
1.40	0.96655	1.1477	0.9771	1.42246	1.2673	0.9727	1.79058	1.3640	0.9739
1.50	0.98824	1.1537	0.9748	1.45695	1.2807	0.9696	1.83660	1.3850	0.9707

Table B.1: (continued)

Bi	<i>Plate</i>			<i>Cylinder</i>			<i>Sphere</i>		
	λ_1	A_1	D_1	λ_1	A_1	D_1	λ_1	A_1	D_1
1.60	1.00842	1.1593	0.9726	1.48917	1.2934	0.9665	1.87976	1.4052	0.9674
1.70	1.02725	1.1645	0.9703	1.51936	1.3055	0.9633	1.92035	1.4247	0.9640
1.80	1.04486	1.1695	0.9680	1.54769	1.3170	0.9601	1.95857	1.4436	0.9605
1.90	1.06136	1.1741	0.9658	1.57434	1.3279	0.9569	1.99465	1.4618	0.9570
2.00	1.07687	1.1785	0.9635	1.59945	1.3384	0.9537	2.02876	1.4793	0.9534
2.20	1.10524	1.1864	0.9592	1.64557	1.3578	0.9472	2.09166	1.5125	0.9462
2.40	1.13056	1.1934	0.9549	1.68691	1.3754	0.9408	2.14834	1.5433	0.9389
2.60	1.15330	1.1997	0.9509	1.72418	1.3914	0.9345	2.19967	1.5718	0.9316
2.80	1.17383	1.2052	0.9469	1.75794	1.4059	0.9284	2.24633	1.5982	0.9243
3.00	1.19246	1.2102	0.9431	1.78866	1.4191	0.9224	2.28893	1.6227	0.9171
3.50	1.23227	1.2206	0.9343	1.85449	1.4473	0.9081	2.38064	1.6761	0.8995
4.00	1.26459	1.2287	0.9264	1.90808	1.4698	0.8950	2.45564	1.7202	0.8830
4.50	1.29134	1.2351	0.9193	1.95248	1.4880	0.8830	2.51795	1.7567	0.8675
5.00	1.31384	1.2402	0.9130	1.98981	1.5029	0.8721	2.57043	1.7870	0.8533
6.00	1.34955	1.2479	0.9021	2.04901	1.5253	0.8532	2.65366	1.8338	0.8281
7.00	1.37662	1.2532	0.8932	2.09373	1.5411	0.8375	2.71646	1.8673	0.8069
8.00	1.39782	1.2570	0.8858	2.12864	1.5526	0.8244	2.76536	1.8920	0.7889
9.00	1.41487	1.2598	0.8796	2.15661	1.5611	0.8133	2.80443	1.9106	0.7737
10.00	1.42887	1.2620	0.8743	2.17950	1.5677	0.8039	2.83630	1.9249	0.7607
12.00	1.45050	1.2650	0.8658	2.21468	1.5769	0.7887	2.88509	1.9450	0.7397
14.00	1.46643	1.2669	0.8592	2.24044	1.5828	0.7770	2.92060	1.9581	0.7236
16.00	1.47864	1.2683	0.8541	2.26008	1.5869	0.7678	2.94756	1.9670	0.7109
18.00	1.48830	1.2692	0.8499	2.27556	1.5898	0.7603	2.96871	1.9734	0.7007
20.00	1.49613	1.2699	0.8464	2.28805	1.5919	0.7542	2.98572	1.9781	0.6922
25.00	1.51045	1.2710	0.8400	2.31080	1.5954	0.7427	3.01656	1.9856	0.6766
30.00	1.52017	1.2717	0.8355	2.32614	1.5973	0.7348	3.03724	1.9898	0.6658
35.00	1.52719	1.2721	0.8322	2.33719	1.5985	0.7290	3.05207	1.9924	0.6579
40.00	1.53250	1.2723	0.8296	2.34552	1.5993	0.7246	3.06321	1.9942	0.6519
50.00	1.54001	1.2727	0.8260	2.35724	1.6002	0.7183	3.07884	1.9962	0.6434
60.00	1.54505	1.2728	0.8235	2.36510	1.6007	0.7140	3.08928	1.9974	0.6376
80.00	1.55141	1.2730	0.8204	2.37496	1.6013	0.7085	3.10234	1.9985	0.6303
100.00	1.55525	1.2731	0.8185	2.38090	1.6015	0.7052	3.11019	1.9990	0.6259
200.00	1.56298	1.2732	0.8146	2.39283	1.6019	0.6985	3.12589	1.9998	0.6170
∞	1.57080	1.2732	0.8106	2.40483	1.6020	0.6917	3.14159	2.0000	0.6079

References

- [1] L. Lamport, R. Shostak, and M. Pease. “The Byzantine Generals Problem.” *ACM Transactions on Programming Languages and Systems*, **4**(3), 1982, pp. 382–401.
- [2] M. J. Fischer, N. A. Lynch, and M. S. Paterson. “Impossibility of Distributed Consensus with One Faulty Process.” *Journal of the ACM*, **32**(2), 1985, pp. 374–382.
- [3] C. Dwork, N. Lynch, and L. Stockmeyer. “Consensus in the Presence of Partial Synchrony.” *Journal of the ACM*, **35**(2), 1988, pp. 288–323.
- [4] Helius. *A Complete History of Solana Outages: Causes and Fixes*. 2024. URL: <https://www.helius.dev/blog/solana-outages-complete-history>.
- [5] V. Buterin et al. *Combining GHOST and Casper*. 2020. arXiv: [2003.03052](https://arxiv.org/abs/2003.03052). URL: <https://arxiv.org/abs/2003.03052>.
- [6] B. Monnot. *Visualising the 7-block Reorg on the Ethereum Beacon Chain*. 2022. URL: <https://barnabe.substack.com/p/pos-ethereum-reorg>.
- [7] Offchain Labs. *Post-Mortem Report: Ethereum Mainnet Finality (05/11/2023)*. 2023. URL: <https://medium.com/offchainlabs/post-mortem-report-ethereum-mainnet-finality-05-11-2023-95e271dfd8b2>.
- [8] E. Buchman, J. Kwon, and Z. Milosevic. *The Latest Gossip on BFT Consensus*. 2018. arXiv: [1807.04938](https://arxiv.org/abs/1807.04938). URL: <https://arxiv.org/abs/1807.04938>.
- [9] Cosmos Hub. *Cosmos Hub v17.1 Chain Halt — Post-mortem*. 2024. URL: <https://forum.cosmos.network/t/cosmos-hub-v17-1-chain-halt-post-mortem/13899>.
- [10] Mysten Labs. *Sui Mainnet Outage Resolution*. 2024. URL: <https://blog.sui.io/sui-mainnet-outage-resolution/>.
- [11] Team Rocket, M. Yin, K. Sekniqi, R. van Renesse, and E. G. Sirer. *Scalable and Probabilistic Leaderless BFT Consensus through Metastability*. 2019. arXiv: [1906.08936](https://arxiv.org/abs/1906.08936). URL: <https://arxiv.org/abs/1906.08936>.
- [12] S. Bano, A. Sonnino, M. Al-Bassam, S. Azouvi, P. McCorry, S. Meiklejohn, and G. Danezis. “SoK: Consensus in the Age of Blockchains.” In: *Proc. 1st ACM Conf. Advances in Financial Technologies (AFT)*. 2019, pp. 183–198.
- [13] C. Cachin and M. Vukolić. *Blockchain Consensus Protocols in the Wild*. 2017. arXiv: [1707.01873](https://arxiv.org/abs/1707.01873). URL: <https://arxiv.org/abs/1707.01873>.
- [14] A. Gervais, G. O. Karame, K. Wüst, V. Glykantzis, H. Ritzdorf, and S. Capkun. “On the Security and Performance of Proof of Work Blockchains.” In: *Proc. ACM Conf. Computer and Communications Security (CCS)*. 2016, pp. 3–16.

- [15] S. Gilbert and N. Lynch. “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services.” *ACM SIGACT News*, **33**(2), 2002, pp. 51–59.
- [16] M. Castro and B. Liskov. “Practical Byzantine Fault Tolerance.” In: *Proc. 3rd USENIX Symp. Operating Systems Design and Implementation (OSDI)*. 1999, pp. 173–186.
- [17] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. “HotStuff: BFT Consensus with Linearity and Responsiveness.” In: *Proc. ACM Symp. Principles of Distributed Computing (PODC)*. 2019, pp. 347–356.
- [18] V. Buterin and V. Griffith. *Casper the Friendly Finality Gadget*. 2017. arXiv: [1710.09437](https://arxiv.org/abs/1710.09437). URL: <https://arxiv.org/abs/1710.09437>.
- [19] I. Amores-Sesar, C. Cachin, and P. Schneider. *An Analysis of Avalanche Consensus*. 2024. arXiv: [2401.02811](https://arxiv.org/abs/2401.02811). URL: <https://arxiv.org/abs/2401.02811>.
- [20] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou. “A Survey of Distributed Consensus Protocols for Blockchain Networks.” *IEEE Communications Surveys & Tutorials*, **22**(2), 2020, pp. 1432–1465.